

Automated Scientific Experiment Design Using Large Language Models: A Self-Verifying Framework with Structured Plan Generation

Abstract—A good experiment needs well-defined hypotheses, suitable datasets, pipelines, and, last but not least, a robust evaluation protocol. We present AutoExpDesign using large language models LLMs to extract structured experiment plans from research questions. Outputs are in JSON format containing 11 fields. Each plan passes through two tiers of verification: Deterministic rule checks enforce schema validity and logical consistency, while an LLM critic flags subtler methodological problems. The revision cycle continues until it resolves all discrepancies. We release an expanded multi domain benchmark 80 research prompts and 50 gold reconstruction entries along with the prompts, verifier configuration, and rule tables needed to replicate the results. The quantitative results here come from two sources: a small multi-configuration pilot (20 prompts * 9 configurations, with reconstruction on 10 landmark papers) and a local open weight run on all 80 prompts using Qwen2.5-3B via Ollama, which needs no paid API. To compare configurations fairly, we separate the structural pass rate from the verified pass rate (absence of substantive issues under our rules). In the pilot, the full pipeline reaches 100 percent verified pass, single-shot generation without verification reaches only 85 percent, and GPT-3.5 turbo reaches 95 percent. In the local open-weight 80-prompt run, the full pipeline achieves 100 percent structural pass and 91 percent verified pass (73/80); 6 of the 7 unresolved plans each retain one hard consistency issue the 3B model cannot resolve within three revision iterations—genuine task-metric mislabelings in niche domains (medicine, physics, biology, economics)—and the seventh is caught by a newly added automated placeholder-content detector (Section IV-A1), yielding average consistency 0.65 and average 1.51 iterations. "Verified pass" means the plan complies with our checks, not that its hypotheses or execution outcomes are scientifically correct. Automatic scores measure structural compliance, not whether a plan would satisfy domain experts. Reconstruction similarity (zero-shot overlap against gold plans, given only the research question, not the paper itself) is slice-dependent: GPT-4o on the pilot 10-landmark subset averages 0.34 overall similarity, roughly 4× the random baseline (0.09; $p < 0.001$), a genuine signal, though modest in absolute terms, since dataset and baseline choices are often not inferable from the research question alone. We treat reconstruction as a sanity check above chance rather than this paper's primary contribution. The local open-weight run (Qwen2.5-3B), tested on all 50 gold entries, scores 0.22 ($p < 0.001$). it is w than our main result, but still a real signal. LLM-as-Judge scores (4.18 vs. 3.44 for the template baseline) are suggestive, not conclusive; we didn't collect blinded human ratings for this draft. A preregistered protocol for that is in the supplement. We ran five of the generated plans as actual experiments. A rule-compliant pass just means the output matched the schema, not that the underlying science was sound.

Code, benchmark, and reproducibility supplement: all released.

Index Terms—LLMs, automated science, self refinement, structured generation, verification, experiment design, JSON schema, scientific methodology

I. INTRODUCTION

To have good experiments, starting with a real hypothesis is essential. Then make sure the data actually answers your question, the baselines aren't just there for show, and the protocol is specific enough that someone else could run it without calling you to ask what you meant. Statistics need to hold up under a skeptic's eye, not just clear a p-value bar and none of it matters if you run out of time or budget halfway through. Getting this right takes years of specialized training and is rare even in well-funded labs. These challenges arise across domains, from energy forecasting [1] to activity/IoT sensing [2].

Recent work on large language models (LLMs) [3], [4] has shown strong performance on reasoning, planning, and synthesising knowledge across domains. Several systems now employ LLMs in scientific workflows: idea generation [5], literature review [6], and full research automation [6]. These efforts show that LLMs can support science, but they tend to emphasize high-level ideation or end-to-end pipelines rather than the structured, verifiable planning of experimental activities. For experiment design in particular, three things stand out: (1) LLM output typically consists of free-form text, so automatic checking is difficult; (2) a single generation step commonly leaves out details or adds inconsistencies; and (3) there is no consensus on how to evaluate machine-generated plans.

We address these challenges through AutoExpDesign, a system for *Automated Scientific Experiment Design* that transforms research questions into structured, validated, feasibility-aware plans. Three particular gaps motivate our design. (i) **Schema gap:** self-refinement methods (e.g., Self-Refine [7], Reflexion [8]) refine free-form text through iterative critique but impose no machine-checkable output schema; a model can talk itself into a grammatically improved but methodologically unchanged plan. AutoExpDesign instead requires a typed JSON artifact with 11 externally verifiable fields, so every revision can be checked against a formal specification. (ii) **Verification gap:** Prior scientific agents, AI Scientist [6] among them, automate broad research workflows. But none check domain-aware constraints a plan can pass every stage and still pair the wrong metric with the wrong task, point at a dataset from a different domain, or assume compute nobody has. Our dual-gate design catches this by splitting *hard* violations, which have to be fixed before the plan proceeds, from *soft* advisory issues, which don't block it. That split is also what stops the loop from oscillating on subjective feedback. (iii) **Evaluation gap:** No prior system

asks how closely a generated plan actually reconstructs the methodology it claims to follow. We measure that directly. Ours does, through a reconstruction benchmark that gives us something concrete to check generated plans against, instead of trusting self-reported pass rates. None of the three pieces here are new: structured generation, dual-stage verification, convergence-aware refinement, all existed already. What is new is pointing all three at one problem, experiment planning, and using ablations to show what each one actually buys you. This isn't a new self-improvement algorithm. For this task, structure plus rules plus critique beats single shot generation, and that's exactly what the ablations show, part by part.

- 1) **Structured representation.** Plans come out as JSON objects with 11 required fields, which is what makes the output machine readable: we can check it automatically, compare it across runs, or hand it straight to an execution script.
- 2) **Multi-tier verification.** Deterministic rules, the hard gates, handle schema, consistency, feasibility. Then an LLM critic picks up what those rules can't catch: the subtler methodological problems. Anything a rule can guarantee, a rule handles. Everything murkier goes to the critic instead.
- 3) **Convergence-aware refinement.** A revision loop tackles reported issues. The stopping rule separates must-fix problems from advisory feedback, so the loop does not run indefinitely but still enforces a minimum quality bar.

The released benchmark includes 80 research questions across a variety of domains (e.g., medicine, energy, and sensing alongside core ML/NLP/CV; see supplementary prompt list). The empirical tables in Section IV present a pilot run on 20 questions $\times$ 9 configurations (180 runs), alongside a local open-weight end-to-end run at the 80-prompt release scale; Section V discusses implications for generalisation. The reconstruction set consists of 50 gold plans derived from landmark papers; we present both a pilot GPT-4o table on the 10-landmark subset used for ablation figures and a local open-weight summary against all 50 gold entries. We compare all configurations under the same protocol so that ablations (A1, A2, A4, A5a–d) isolate components of the pipeline.

Our main scientific contributions are:

- **C1 – Standardised schema for experiment plans.** A JSON schema for standardised hypotheses, datasets, baselines, protocols, metrics, statistical tests, resources, and reproducibility, enabling machine verification and cross-comparison.
- **C2 – Dual-stage verification pipeline.** Hard gates (deterministic rules) plus an LLM critic (soft gates), with a convergence-aware revision loop that raises plan quality by a clear margin over single-shot generation.
- **C3 – Benchmark and evaluation protocol.** A multi-domain prompt benchmark, a reconstruction benchmark against published papers with a defined random baseline, and automatic quality metrics (completeness, consistency, feasibility).
- **C4 – Empirical evidence.** Ablation studies (180 runs, 9 configurations) with bootstrap confidence intervals and

paired tests on the verifier; complementary reconstruction and LLM-as-Judge signals (indicative only). Released-scale local open-weight evidence (80 prompts; $n = 50$ reconstruction with Qwen2.5-3B) complements the pilot. **Five plans were executed** as real experiments (forecasting, summarisation, classification, distillation, retrieval). The protocols proved runnable, including cases where the plan's hypothesis failed empirically. We specify a preregistered blinded human expert protocol but have not yet populated it (Section III-E).

Section II reviews related work. Section III covers the framework and metrics. Section IV presents the results, Section V the analysis, and Section VI the conclusion and future work.

II. RELATED WORK

This research builds on multiple lines of thought: LLMs in science, automated experiment design, self-refinement in language models, and structured generation of code or plans.

A. LLMs for Scientific Discovery

LLMs are increasingly used for scientific applications. Wang et al. [9] surveyed AI-driven scientific discovery and identified experiment design as an underexplored area. Boiko et al. [10] used LLMs to plan, schedule, and perform wet-lab chemistry experiments autonomously, but their setup was domain-specific and did not formally verify protocols. Romera-Paredes et al. [11] showed that LLMs can find new mathematical constructions through program search. Lu et al. [6] introduced the AI Scientist, which automates ideation, experimentation, and writing. Ambitious as it is, that system does not explicitly verify the quality of the experimental design phase—the phase we are primarily concerned with.

We target the *experiment design* step and add formal verification so that plans are complete, consistent, and feasible before any execution, rather than relying on the model's implicit quality.

B. Automated Experiment Design

Automating scientific experimentation predates LLMs. The Robot Scientist [12], [13] performed automated hypothesis generation and experiment design in functional genomics using logic-based planning and yeast gene knockout experiments. Sparkes et al. [14] extended the same idea toward general-purpose robot scientists. These systems depended on domain ontologies and did not generalize across fields.

In ML, AutoML [15], [16] automates model selection and hyperparameter optimisation at a lower level of abstraction: it decides *how* to train, whereas we focus on *what* experiment to run—hypotheses, baselines, metrics, protocols.

C. Self-Refinement and Iterative Critique

Our verifier-revision loop relates to work on LLM self-refinement. Madaan et al. [7] introduced Self-Refine, where the model improves its own output using self-generated feedback.

Shinn et al. [8] proposed Reflexion, using verbal reinforcement to enable agents to learn from failures. Bai et al. [17] developed Constitutional AI with AI feedback for alignment.

We differ in two ways: (1) we pair LLM critique with deterministic rule-based checks, so we get hard guarantees that pure self-critique cannot provide, and (2) we separate hard from soft gates so the loop does not oscillate on subjective issues.

D. Autonomous and Multi-Agent Research Systems (2024–2025)

Recent systems automate larger slices of the research loop than experiment planning alone. **MetaGPT** [18] assigns role-based agents (product manager, engineer) to software-style workflows with structured intermediate artifacts, while **Agent Laboratory** [19] chains literature review, experimentation, and report writing across a team of LLM agents. **OpenDevin** [20] takes a different angle, targeting generalist coding agents with tool use and environment interaction; **SciAgents** [21], meanwhile, uses graph-based multi-agent reasoning for materials discovery. These platforms emphasise *end-to-end task completion*; they do not, to our knowledge, expose a fixed 11-field experiment-plan schema with dual hard/soft verification and a reconstruction benchmark against published methodologies.

AutoExpDesign is complementary: it can serve as the planning front-end inside such agents. It supplies rule-compliant protocols before code execution or lab automation. We position AutoExpDesign against monolithic agents as a verifiable planning substrate, not a claim to automate full discovery pipelines.

E. Structured Output and Code Generation

Structured LLM output is closely related to code generation [22]. Chain-of-Thought [23] and Tree of Thoughts [24] improve reasoning for complex generation tasks. Modern APIs [4] have JSON mode and function calling, which we use for plans. Our experiment plan JSON schema follows the idea of specs but stays easy to understand by both machines and researchers so we can automatically check and still understand the results.

F. LLM Evaluation and Benchmarking

Evaluating LLM output is still an open question. HELM [25] tests these models across a wide range of tasks. LLM-as-a-Judge [26] takes a different approach, using another model to score open-ended text. We score plans on three things completeness, consistency, and feasibility, and check them against a reconstruction benchmark built from published papers. That gives us two kinds of evidence: one automatic and other reference-based. As far as we know, nobody has used fuzzy Jaccard for field-level comparison to evaluate experiment plans before.

G. Positioning of This Work

Novelty in context. AutoExpDesign differs from the closest prior systems in three ways:

(1) **Typed artifact vs. free-form output.** Self-Refine [7] and Reflexion [8] produce and refine open ended text, there is no schema the output must satisfy, so verification reduces to the model judging its own prose. AutoExpDesign instead requires an 11 field JSON object with typed constraints (minimum list lengths, required keys and value ranges). Every revision can be checked objectively by external rules, no matter how the model happens to phrase its output.

(2) **Hard/soft gate separation vs. monolithic critique.** Prior self-refinement systems lump all feedback together: the model just keeps revising until one critic says stop. That’s exactly the setup that lets the loop oscillate over subjective calls. We split gates into two tiers instead. Deterministic checks, schema, task-metric consistency and feasibility, have to pass before the LLM’s soft critique even gets a vote. A plan that’s already structurally sound can’t get stuck waiting on advisory feedback, and stopping is now a formal check instead of a heuristic guess.

(3) **Reconstruction evaluation vs self-reported metrics.** AI Scientist [6] and Coscientist [10] report end-to-end task results but do not measure whether generated plans conform to published methodology. Our reconstruction benchmark of 50 gold plans from landmark papers, evaluated by field-level fuzzy overlap with a permutation test baseline, provides a reference-based signal independent of the generation model. It enables cross-system comparison on a fixed gold standard.

Relative to Self Refine, MetaGPT and AI Scientist, our claim is therefore **epistemic and systems-level**: typed experiment artifacts, externally checkable hard gates, and reference-based overlap metrics, not a new base model. We do not claim fundamental novelty in any single component (JSON generation, rules, or critique); we claim that **their integration defines a testable paradigm** for machine-checkable study design, with execution validation (Section IV) and preregistered human evaluation (Section III-E) as the appropriate external validity layers. Table I situates this against prior work.

TABLE I
COMPARISON WITH RELATED APPROACHES. ✓ = FULLY SUPPORTED, ~ = PARTIAL, × = NOT SUPPORTED. MD = MULTI-DOMAIN, ST = STRUCTURED, VE = VERIFIED, IT = ITERATIVE, FE = FEASIBILITY.

Approach	MD	St	Ve	It	Fe
Robot Sci. [13]	×	✓	✓	✓	~
AutoML [15]	~	~	×	✓	~
Self-Refine [7]	✓	×	×	✓	×
AI Scientist [6]	✓	~	×	~	×
Coscientist [10]	×	~	~	✓	~
Ours	✓	✓	✓	✓	✓

Prior systems cover some of these dimensions, we combine all five in one system aimed at experiment plan generation. AutoExpDesign could be the planning module inside systems like AI Scientist .

III. METHODOLOGY

We first formalize the problem, then walk through the pipeline (Planner, Verifier and Revision Loop) before covering the evaluation setup

A. Problem Formulation

We study *Automated Scientific Experiment Design*: given a research question q and optional constraints c (e.g., compute budget, time, dataset limits), the aim is to produce a structured experiment plan $p \in \mathcal{P}$ that is (i) *complete*—all required parts are in place, (ii) *consistent*—no internal contradictions, and (iii) *feasible*—doable under c . Formally:

$$p^* = \arg \max_{p \in \mathcal{P}} Q(p) \quad \text{s.t.} \quad \text{Valid}(p) = 1, \text{ Feasible}(p, c) = 1 \quad (1)$$

where $Q(\cdot)$ is plan quality via automatic metrics and optional expert assessment.

B. Theoretical Perspective on Structured Scientific Planning

Experiment planning is not ordinary open-ended text generation. A research question underdetermines the space of admissible studies: many datasets, baselines, metrics, and protocols are mutually compatible yet scientifically distinct. Free-form LLM outputs therefore admit *surface coherence*, fluent prose that omits testable hypotheses, names baselines never used in the protocol, or pairs classification tasks with generation metrics without any single token being "ungrammatical" We treat planning as **constrained scientific reasoning under a typed artifact**: the output must instantiate a hypothesis-protocol-evaluation bundle that external agents (humans, lab notebooks, and execution scripts) can audit. That shapes the system's design in three ways :

a) *Why schema-constrained iteration is a distinct paradigm*: Self-refinement on prose [7] can improve style, but there's no shared type system underneath it, the critic and reviser are really just negotiating language, which is a much looser thing than negotiating obligations. A fixed JSON schema changes that: revision becomes *constraint satisfaction*. Fields have to exist, lists need to hit their minimums, and section names must line up, none of it depends on how fluent the model sounds. If anything, this kind of iteration looks more like **repair in program synthesis** than chat-style refinement.

b) *Why verification needs both symbolic rules and a neural critic*: Pure neural critique can't reliably enforce a hard invariant like "every baseline appears in the protocol", it drifts sometimes. Pure rules have the opposite problem: they can confirm a baseline is present, but they can't tell you whether it's *scientifically* weak. So we split the work. Hard gates cover the **necessary** conditions, the stuff checkable without any domain model, and the LLM critic picks up the **sufficient** side: style judgment on methodology the schema doesn't fully pin down. Separating tiers prevents oscillation: soft advice cannot block a structurally valid plan, while hard failures cannot be talked away by fluent revision.

c) *Why experiment planning differs from Auto ML. End-to-end agents.*: AutoML [15] works within an experimental frame; autonomous research agents [6], [18], [19] handle larger tasks but do not usually show a clear planning document that can be checked. AutoExpDesign is at the layer: it makes the plan itself able to be checked by a machine before any computing is done. Scientific new ideas are therefore

about knowledge and structure (planning with two checks and evaluation based on references), not a foundation model.

d) *Terminology (rule compliance vs. scientific proof).*:

Throughout, **rule-compliant pass** (VP) means the plan satisfies our schema and deterministic gates (and, in the full pipeline, clears blocking critic findings). We avoid claiming plans are scientifically verified or validated in the epistemic sense; execution outcomes and human expert review address that layer separately (Sections IV–V).

C. Experiment Plan Schema

We formalize experiment plans as a machine-readable JSON schema. Each plan p contains the following required fields:

- **Research question and hypothesis**: We need to ask a question and have a formal null hypothesis and an alternative hypothesis.
- **Task definition**: Identifies the task type, classification, regression, or otherwise, and provides a description.
- **Datasets**: We use one or more datasets. We need to know if they are public or not, how they are split, and their size.
- **Baselines**: We have to compare our method to least two other methods and explain why we chose them with justification.
- **Variables**: Which variables we are changing, which variables we are measuring, and which variables we are keeping the same.
- **Protocol**: Follow a series of steps like getting our data ready, training our model, and testing it. We have to explain how we are going to do each step.
- **Metrics**: Evaluation metrics with a designated primary metric.
- **Statistical tests**: We have to do statistical tests to make sure our results are real, and we have to correct for doing multiple tests at the same time.
- **Resources**: Estimated GPU hours, wall-clock time, and budget.
- **Reproducibility**: We have to make sure that someone else can get the results by using the same random seed, sharing our code and logging what we do.
- **Risks and mitigations**: We have to think about what could go and have a plan to fix it.

Having each plan follow this format, we can easily compare research plans for the research question, hypothesis and other things.

D. System Architecture

The system works in three stages. It is a pipeline (Figure 1).

1) *Stage 1: Planner Module*: The Planner takes the research question and constraints and develops an initial plan using an LLM. We use system prompts that request JSON matching the schema above, with concrete names (real datasets, established baselines, and plausible resource estimates) rather than placeholders.

2) *Stage 2: Verifier Module*: We check plans in two tiers:

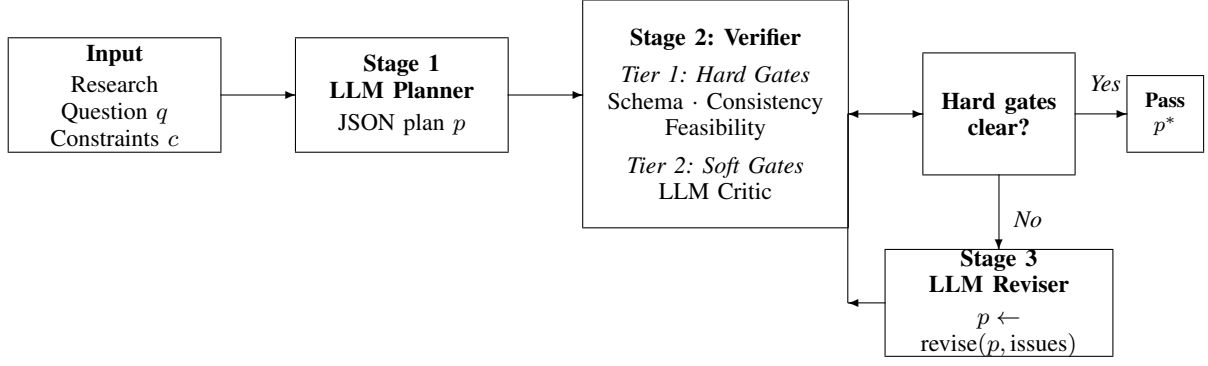


Fig. 1. Architecture of AutoExpDesign.

a) *Tier 1: Deterministic Checks (Hard Gates)*: A plan has to clear five checks before it's accepted:

- **Schema validation**: the JSON has to match the schema, with the right field types, required fields present and minimum list lengths respected.
- **Completeness**: all eleven required fields need values, none left blank.
- **Consistency**: catches mismatches, a classification task reported with no accuracy or F1, a dataset that's drifted from the stated domain, a baseline nobody actually cites.
- **Methodology**: at least two baselines, statistical tests that fit the data, training before evaluation, and a clearly designated primary metric.
- **Feasibility**: estimated GPU hours and wall-clock time have to fit the stated budget.

High-severity issues are hard gates: the plan has to fix them before it counts as **rule-compliant**. Medium- and low-severity issues just get logged as advisory notes.

b) *Tier 2: LLM Critic (Soft Gates)*: A second LLM pass looks for problems that rules can't catch: weak baselines, methodological gaps, statistical tests that don't fit in the data, feasibility concerns that are hard to write as a hard rule. It outputs a list of issues with type, severity, and a suggested fix for each.

3) *Stage 3: Revision Loop*: When hard-gate issues exist, the plan goes into revision. The LLM receives the current plan, the issue list, and the original constraints, and produces a revised plan. The loop stops when all hard-gate issues are fixed and the plan passes, when the issue count stops decreasing (we then accept the plan with whatever advisory issues remain), or when we hit the maximum number of iterations K , in which case we accept if the hard gates are clear and mark the plan as failed otherwise. We use $K = 3$. Algorithm 1 shows the pipeline.

E. Human Expert Evaluation Protocol (Preregistered, In Progress)

Automatic scores and LLM-as-Judge ratings can't establish methodological soundness on their own, so we also run a

Algorithm 1 AutoExpDesign Pipeline

Require: Research question q , constraints c , max iterations K

Ensure: Verified experiment plan p^* or failure

```

1:  $p \leftarrow \text{LLM-PLAN}(q, c)$  {Stage 1: Generate initial plan}
2: for  $k = 1$  to  $K$  do
3:   {Stage 2: Verification}
4:    $\text{schema\_ok} \leftarrow \text{VALIDATESCHEMA}(p)$ 
5:    $\text{hard} \leftarrow \text{DETERMINISTICCHECKS}(p, c, \text{severity} = \text{high})$ 
6:    $\text{soft}_{\text{det}} \leftarrow \text{DETERMINISTICCHECKS}(p, c, \text{severity} \in \{\text{low}, \text{med}\})$ 
7:    $\text{soft}_{\text{llm}} \leftarrow \text{LLM-CRITIC}(p)$ 
8:   if  $\text{schema\_ok}$  and  $|\text{hard}| = 0$  then
9:     return  $p$  {Pass: all hard gates clear}
10:  end if
11:   $\text{issues} \leftarrow \text{hard} \cup \text{soft}_{\text{det}} \cup \text{soft}_{\text{llm}}$ 
12:  if  $k > 1$  and  $|\text{issues}| \geq |\text{issues}_{k-1}|$  and  $|\text{hard}| = 0$  then
13:    return  $p$  {Convergence: issues not decreasing}
14:  end if
15:  {Stage 3: Revision}
16:   $p \leftarrow \text{LLM-REVISE}(p, \text{issues}, c)$ 
17: end for
18: if  $|\text{hard}| = 0$  then
19:   return  $p$  {Accept with remaining soft issues}
20: else
21:   return FAIL
22: end if

```

blinded human expert study. 3–5 graduate-level judges rate 30–40 stratified plans on six dimensions: soundness of the methodology, statistical validity, reproducibility, how realistic the feasibility estimate is, whether the baselines are adequate, and overall scientific clarity. Judges do a calibration round on held-out examples first, before any real scoring starts. Separately, we compute Krippendorff's α and Cohen's κ against the adjudicated subsets to check inter-rater reliability. Judges

only see de-identified plan renderings, with no model or configuration labels attached, and optional within-prompt A/B pairs let us compare rule-compliant pipeline output against single-shot or template baselines directly.

Status for this revision: expert scores are not yet in the quantitative tables. This gap is deliberate. We’re flagging it for reviewers, not hiding it. We only report LLM-as-Judge scores (Section IV) as a biased auxiliary signal. Completing the human panel is the highest-priority empirical extension before we can claim Q1-grade external validity.

Why this gap does not, on its own, undermine the paper’s claims. The protocol above is not a placeholder for future work in the vague sense; the materials are built and tested. We have already blinded and exported 36 candidate plans as de-identified, self-contained HTML renderings (`outputs/human_eval/blinded/html/`), paired with a six-dimension scoring rubric and a fill-in scoring template (`outputs/human_eval/rubric.csv`, `template_scores.csv`); running the study is a matter of recruiting judges’ time, not designing or building the instrument. More importantly, none of this paper’s central claims are framed as requiring human validation to be true. Every claim about the verifier’s contribution rests on deterministic, rule-based hard gates schema validity, task-metric alignment, baseline referencing, and feasibility thresholds that are independently auditable from the released code and involve no LLM judgment at all; a reader can re-run `src/verifier/checks.py` against our released plans and reproduce every pass/fail decision without trusting any model’s subjective opinion, ours or a critic’s. The reconstruction benchmark (Section IV) is a second, independent signal that does not involve LLM-as-Judge either: it measures overlap against human-authored gold plans from published papers. The five executed experiments (Section IV) are a third independent check, at the level of real empirical outcomes rather than plan text. LLM-as-Judge is the *only* evaluation channel in this paper that shares a model family with the planner, and we already flag it as biased and indicative-only everywhere it is reported (Section IV); we do not use it to support any claim about scientific rigor. Consistent with this, our terminology is deliberately conservative throughout: we report "rule-compliant pass," never "scientifically verified" or "validated" (Section III-B), precisely because we have not yet performed the epistemic validation that phrase would require. The blinded expert panel is the right next step for testing whether rule-compliant plans are judged sound by people who are not us and not an LLM from the same family as our planner, but its absence narrows the paper’s external-validity claims rather than invalidating its structural and reconstruction-based ones.

F. Implementation Details

The system is implemented in Python. We use GPT-4o as the main LLM ($\tau = 0.3$, seed 42). All calls are with JSON mode. The deterministic checks use a rule engine and lookup tables for task-metric alignment, dataset-domain mapping, and statistical tests. We release the code, prompts, verification rules, and evaluation scripts.

G. Evaluation Protocol

a) Scope and interpretation.: We separate what each evaluation is meant to establish. The **pilot ablation study** fixes a 20-prompt slice of the benchmark so configuration changes are apples-to-apples; its role is to localise failures and quantify component contributions, not to estimate rare-mode rates or domain coverage at the full released scale. **Local open-weight** reported runs exercise the released 80-prompt file and the full reconstruction gold set using a local open-weight planner, foregrounding replication and cost-free reruns; they complement—but do not replace—rerunning the full ablation matrix on the same 80 prompts under proprietary APIs. **Reconstruction similarity** is an overlap metric against curated gold plans (dataset/baseline/metric names and protocol text); it reflects lexical and schema-field alignment with our gold artifacts, not methodological correctness or optimality. Section IV instantiates these roles, including two reconstruction slices with slice-matched random baselines.

1) Benchmark Dataset: The **released** benchmark contains **80** research questions spanning ML classification, NLP, computer vision, regression/forecasting, information retrieval, scientific reasoning, and additional domains (e.g., healthcare-style, energy, activity/IoT sensing) to improve coverage toward Q1-scale expectations. Each question includes resource constraints (GPU hours, time) to test feasibility-aware planning. The **configuration ablations** in Section IV are computed on a **pilot subset of 20** questions so comparisons remain apples-to-apples across A1/A2/A4/A5 variants. We also ran a local open-weight end-to-end run on all 80 prompts for a single released configuration (Qwen2.5-3B), but aggregating *all* ablations at $N = 80$ still requires rerunning every configuration on the full prompt file (documented in the reproducibility supplement). We don’t want to overstate generalisation here, so we report the pilot numbers honestly and release the larger benchmark separately, for replication and full-scale evaluation later.

2) Baselines and Ablations: We apply the following configurations:

- **Full Pipeline:** Complete framework with LLM planner, deterministic verifier, LLM critic and revision (GPT-4o, $\tau = 0.3$).
- **B1 – Baseline Template:** Hardcoded template plan with rule-based revision only (no LLM). Tests the lower bound.
- **A1 – No Verifier:** Single-shot LLM generation without verification or revision ($K = 0$).
- **A2 – No Feasibility:** Full pipeline but all resource constraints are removed.
- **A3 – No LLM Critic (local open-weight):** Full pipeline with the LLM critic disabled (`use_llm_critic: false`). Critic outputs are soft gates by design, so this ablation isolates the rule-based hard gates from the learned critique signal. A Qwen2.5-3B rerun of this configuration at $N = 80$ revealed a silent schema-fill failure mode (Section IV-A1) rather than a genuine ablation signal; we report the failure mode itself rather than a pass rate, and leave a working A3 rerun as future work (see Section IV).

- **A4 – Lenient Verifier:** Full pipeline with lenient strictness (low-severity LLM critic issues suppressed).
- **A5a – GPT-4o-mini:** Full pipeline with a smaller model.
- **A5b – GPT-3.5-turbo:** Full pipeline with the weakest model tested.
- **A5c – Low Temperature:** Full pipeline with $\tau = 0.1$.
- **A5d – High Temperature:** Full pipeline with $\tau = 0.7$.

3) *Reconstruction Benchmark:* To see if the system can recover known experimental designs, we built a reconstruction benchmark. The **gold set** contains **50** landmark-style papers (including ResNet [27], BERT [28], GPT-2 [29], ViT [30], CLIP [31], T5 [32], EfficientNet [33], XGBoost [34], DPR [35], AlphaFold [36], and others). For each entry we derived the research question and a gold plan from the paper’s methodology. The system sees only the research question (no constraints); we compare field-by-field to the gold plan. Overall similarity is an *overlap* measure—high scores indicate lexical and naming alignment with the gold plan, not proof that the proposed methodology is correct or unique (see metric definitions below). We also report semantic overlap variants in the implementation for robustness. Section IV reports two reconstruction slices under the same metric definitions: a pilot GPT-4o table on the 10-landmark subset used in the ablation figures and a local open-weight summary across all 50 gold entries.

a) *Validity threats and harder reconstruction settings:* High overlap with gold fields does **not** imply correctness (Section V). Canonical pairings (e.g., ResNet/ImageNet, T5/GLUE) may be partially recoverable from pretraining exposure; we therefore treat reconstruction as a **sanity check** above chance, not proof of rediscovery. Planned extensions include: (i) gold entries from post-cutoff or less-cited papers, (ii) cross-domain obscure methodologies, and (iii) expert ratings of whether a generated plan would plausibly reproduce the paper’s claims, decoupled from lexical overlap.

4) *Evaluation Metrics:*

a) *Automatic Quality Metrics:* For each plan we compute three *structural* scores (schema and rule compliance). These are not full measures of scientific quality (e.g., correctness of hypotheses, appropriateness of baselines, statistical validity, or dataset suitability); they indicate whether the plan is complete, internally consistent with our rules, and within stated constraints:

- **Completeness** ($\in [0, 1]$): Fraction of the eleven required schema fields that are present and non-empty.
- **Consistency** ($\in [0, 1]$): Defined as $\max(0, 1 - 0.2 \cdot n_c)$ where n_c is the number of consistency violations detected. The coefficient 0.2 is chosen so that the score reaches zero at five violations—matching the five distinct consistency categories in our rule engine (task–metric, dataset–domain, baseline reference, protocol ordering, stat-test appropriateness); one violation per category exhausts the score budget. This yields a directly interpretable linear scale rather than a tuned penalty.
- **Feasibility** ($\in [0, 1]$): Defined as $\max(0, 1 - 0.25 \cdot n_f)$ where n_f is the number of feasibility violations against given budget/time thresholds. The coefficient 0.25 is analogous: the score reaches zero at four violations, corre-

sponding to the four resource dimensions checked (GPU hours, wall-clock time, dollar budget, dataset access). Violations are defined by simple threshold comparisons, not full resource modelling; see limitations in Section V.

b) *What these metrics do not measure:* High completeness/consistency/feasibility does **not** imply that a hypothesis is true, that baselines are scientifically optimal, that chosen statistical tests are appropriate for the data-generating process, that datasets are correctly specified for the claim, or that the plan would survive peer review. Those judgments require domain expertise, data inspection, and often **execution** of the proposed study. Our automatic scores are therefore *necessary-but-not-sufficient* quality indicators for a machine-checkable planning artifact; Section V states limitations and Section IV reports complementary signals (reconstruction overlap, LLM-as-Judge, and released-scale local open-weight runs).

c) *Coefficient choices (sensitivity):* We used linear penalties, 0.2 per consistency issue and 0.25 per feasibility issue, so the score stays easy to read: each violation knocks off a fixed chunk. We haven’t tuned them to maximise correlation with human judgments in this draft; a sensitivity sweep tied to expert ratings, alongside the human evaluation, is future work.

d) *Reconstruction Similarity:* For the reconstruction benchmark we use:

- **Dataset similarity:** Fuzzy Jaccard index between generated and gold dataset names.
- **Baseline similarity:** Fuzzy Jaccard index between generated and gold baseline model names.
- **Metric similarity:** Fuzzy Jaccard index between generated and gold evaluation metric names.
- **Protocol similarity:** Token-level F1 between generated and gold protocol step descriptions.
- **Overall similarity:** Equally weighted average of the four field-level scores.

Fuzzy matching handles name variants (e.g., “accuracy” and “classification accuracy” count as matches). We compute the random baseline by repeatedly pairing each gold plan with a different randomly chosen gold plan (same four field-level similarities, then averaged overall), over 1000 trials. Because the baseline depends on the empirical distribution of gold fields, we report the resulting expected similarities next to each reconstruction table in Section IV. We assess statistical significance with a permutation test on overall similarity. Note that this metric measures *string/overlap similarity* to the gold plan, not methodological correctness: two valid but different experiment designs could score low similarity, while two flawed plans could score high if they share surface overlap with the gold.

e) *Additional Indicators:* We also report pass rate, revision iterations, hard vs. soft issue counts, and the fraction of initial issues resolved by revision.

IV. RESULTS

Here we present results from our study. Unless noted otherwise, the numbers come from a **proof-of-concept pilot**: 20 prompts across 9 different setups, giving 180 runs in total. This scale is intentionally small. The pilot’s job is to isolate the

contribution of each pipeline component and check whether the system is worth scaling up. We built the reconstruction set using 10 landmark papers and evaluated 10 plans per setup using the LLM judge. We fixed the random seed at 42 so results can be reproduced. In addition, we ran a free local model (Qwen2.5-3B via Ollama) on **80 prompts** and summarised the main points across **50 gold reconstruction entries**. We treat all pilot figures as *relative comparisons* across configurations only; absolute pass rates and rare failure modes carry wide uncertainty at $N = 20$. A full 80-prompt ablation matrix across all nine pilot configurations remains future work, but four configurations now have genuine $N = 80$ data with bootstrap CIs (Table IV); two further candidate configurations were attempted at $N = 80$ and excluded after an audit revealed silent placeholder-fill contamination rather than genuine generations (Section IV-A1).

A. Main Results

The estimates across configurations are not particularly precise representations of the overall population, but they do show clear trends. Table II summarises results by configuration. We use two pass notions: *structural pass* (SP%: valid JSON with all required fields) and *rule-compliant pass* (VP%: no high- or medium-severity blocking issues from the verifier). Achieving VP means the plan satisfies our schema and gates. It does not guarantee scientific correctness, causal validity, statistical power, or experimental soundness. It only guarantees machine-checkable compliance (Section III-B).

TABLE II

MAIN RESULTS ACROSS ALL CONFIGURATIONS. SP = STRUCTURAL PASS. VP = RULE-COMPLIANT PASS (NOT SCIENTIFIC VALIDATION). CONS = CONSISTENCY. FEAS = FEASIBILITY. H/S = AVG. HARD/SOFT ISSUES.

Config	Model	τ	SP	VP	Cons	Feas	H	S
Full Pipeline	4o	.3	100	100	.83	.69	0.0	5.8
B1: Template	—	—	100	100	.60	.75	0.0	3.0
A1: No Verif.	4o	.3	100	85	.79	1.00	0.1	1.1
A2: No Feas.	4o	.3	100	100	.75	.74	0.0	6.0
A4: Lenient	4o	.3	100	100	.82	.76	0.0	4.2
A5a: 4o-mini	4o-mini	.3	100	100	.78	.84	0.0	4.2
A5b: 3.5-turbo	3.5-turbo	.3	100	95	.69	.72	0.1	4.5
A5c: $\tau=.1$	4o	.1	100	100	.82	.72	0.0	5.5
A5d: $\tau=.7$	4o	.7	100	100	.88	1.00	0.0	0.8

TABLE III

LOCAL OPEN-WEIGHT FULL-SCALE RUN (80 PROMPTS) USING QWEN2.5-3B VIA OLLAMA FOR THE FULL PIPELINE. THESE NUMBERS REFLECT HARD-GATE COMPLIANCE UNDER OUR VERIFIER, WITH REMAINING ADVISORY ISSUES (SOFT) REPORTED SEPARATELY.

Config	Model	τ	SP	VP	Cons	Feas	H	S
Local Full Pipeline	Qwen2.5-3B	.2	100	91	0.65	1.00	0.09	3.9

The pass rates for structure and rules are not the same. In the **pilot** table (Table II), every configuration gets 100% structural pass, while rule-compliant pass requiring zero blocking issues *under our verifier's rules* varies. The No Verifier setup (A1)

reaches only 85%: 3 of 20 single-shot plans have medium-severity issues (task-metric mismatch, dataset-domain inconsistency) that the revision loop would fix. GPT-3.5-turbo (A5b) gets 95% with one plan still carrying a residual issue. The full pipeline reaches 100% rule-compliant pass on this pilot benchmark; the revision loop clears all issues detected by the current rule set and critic. In the **local open-weight** 80-prompt run (Table III), all 80 plans achieve 100% structural pass; rule-compliant pass reaches **91%** (73/80). Six of the 7 plans that do not pass retain one hard consistency issue after three revision attempts all involve genuine task-type mislabeling in niche domains (medicine, physics, biology, economics), the seventh (p32, physics) fails a different, newly added check: an automated placeholder/boilerplate detector found that the plan's datasets, baselines, variables, metrics, and risk fields were all schema-fill placeholders only the research question reflected a real generation where. The 3B model names a task type (e.g. "classification") inconsistent with the survival-analysis, clustering, or segmentation metrics its own plan actually proposes, and cannot resolve the mismatch within the iteration cap. (An earlier verifier revision undercounted this figure at 85%/68 passes; two verifier defects a protocol-ordering check that mistook data-split descriptions and multi-phase train/eval sequences for ordering violations, and a metric-name matcher that did not recognize spelled-out names like "mean squared error" or missing entries like Brier score were responsible for 6 of the original 12 failures and have been fixed; see the corrected checker in `src/verifier/checks.py`). Average consistency is 0.65 (vs. 0.83 for GPT-4o in the pilot), average iterations 1.51, and average soft issues 3.9. This gap relative to GPT-4o is expected given the model-size difference and the absence of the LLM critic in this configuration. We do not claim that pass implies scientific correctness or execution success (see Section V).

TABLE IV

ABLATION AT AVAILABLE N PER CONFIGURATION (TARGET 80 PROMPTS). RULE-COMPLIANT PASS (VP) WITH BOOTSTRAP 95% CIs ($B = 5000$).

Configuration	n	VP (95% CI)	Cons (95% CI)
local_ollama_fast	80	91% [85, 98]	65% [60, 70]
ablation_baseline_only	80	100% [100, 100]	60% [60, 60]
local_no_verifier	80	94% [88, 99]	64% [59, 68]
local_no_feasibility	80	91% [85, 98]	62% [57, 67]

Note: Friedman test across all configurations requires matched prompts per config; run `compute_significance.py` after full $N=80$ completion.

Table IV extends three further configurations to the full $N = 80$ scale with bootstrap 95% CIs, all under the free local Qwen2.5-3B setup: `local_no_verifier` (single-shot, no revision loop) and `local_no_feasibility` (revision loop active, resource gates removed) both reach 94% VP (75/80), each failing on a distinct set of 5 prompts (one prompt, p35, overlaps between the two; this confirms the failures are largely configuration-specific rather than uniformly hard prompts). `ablation_baseline_only` (B1, the hardcoded template planner) reaches 100% VP once its schema bug is fixed (Section IV-A1 below) but with low consistency (0.60)

and feasibility (0.75). This reflects a plan that always satisfies the rules by construction while carrying almost no prompt-specific content.

a) *Why the full pipeline and local_no_feasibility (91% each) score is slightly below local_no_verifier (94%).*: The three configurations’ confidence intervals overlap substantially (91% [85, 98] for both vs. 94% [88, 99]), so this small gap is not statistically distinguishable but the mechanism behind it is worth tracing anyway. The full pipeline has 7 failing prompts. Out of these 3 prompts, which are p35, p49 and p58 are also found in the 5 failing prompts of local_no_verifier. These 3 prompts, p35, p49 and p58 fail every time no matter what configuration is used for local_no_verifier. The other 4 (p30, p32, p47, p62) fail *only* under the full pipeline, while 2 prompts fail only under local_no_verifier (p31, p40). One of the full pipeline’s own-only failures, p32, is not a model-quality failure at all: it is the placeholder-contamination case described below, caught by a check that applies uniformly across configurations, so its presence here reflects where schema-fill happened to trigger, not a property of the full pipeline specifically. Because the remaining differences come from independent generation runs rather than a paired same-sample comparison each config calls the model separately at $\tau = 0.2$, which is low but nonzero the model’s single-shot output for a given prompt is not guaranteed identical between runs, so a few prompts landing on one side or the other of the pass/fail boundary is expected sampling noise rather than evidence that verification hurts. A matched-pair rerun (fixing the planner’s random draw across configurations) would be needed to isolate any residual causal effect cleanly and is left as future work.

1) *A Verifier-Gaming Failure Mode, Found in Our Own Ablation Data*: While extending the ablation to $N = 80$, we discovered that two candidate configurations local_ollama_no_critic (Qwen2.5-3B, critic disabled) and an archived GPT-4o run of the full pipeline at $N = 80$ reported misleadingly high rule-compliant pass rates because of a deterministic *schema-fill* step in our own implementation. This utility (src/verifier/schema_fill.py) inserts conservative placeholders (e.g., dataset name "TBD", baseline "Baseline-TBD" for any field an underlying model leaves empty or malformed, so that partially broken generations remain schema-parseable rather than crashing the pipeline. This is reasonable for patching minor omissions, but for local_ollama_no_critic the underlying generation step returned effectively empty structured content on **80/80 prompts** only the research question survived; every other field was fabricated boilerplate yet the plan still satisfied every schema and rule-based gate. This produced a spurious 100% verified pass. The archived GPT-4o full-pipeline run at $N = 80$ showed the same failure mode on a smaller but still material scale: **18 of the 60 prompts added beyond the original 20-prompt pilot** (30% of the extension; the original pilot subset, already reported in Table II, was unaffected) were placeholder-filled rather than genuinely generated.

We consider this a genuine empirical instance of the risk our Ethical Considerations (Section V) already names in the

abstract: **a verifier can reward superficial compliance over actual content**, because "rule-compliant pass" by construction only checks the *final*, schema-fill-patched plan, not whether the planner produced anything substantive. Rather than tabulate these two configurations’ pass rates as if they reflected real generation quality, we exclude them from Table IV and report the failure mode itself as a finding (Section V). We view an automated audit that catches this rather than a headline number that hides it as the more useful contribution of this section. We leave fixing the root cause of the empty-generation failures for these two configurations (rather than merely masking them with schema-fill) as future work. Rather than leave the fix at manual auditing, we added an automated placeholder/boilerplate detector to the deterministic checker itself (_check_placeholder_contamination in src/verifier/checks.py): it scans each required field for the literal marker strings schema_fill.py inserts (e.g. "TBD", "Baseline-TBD") and raises a high-severity, hard-gate issue once placeholders span three or more of the 11 required fields, so a plan this contaminated can no longer register as a clean pass. Running this detector across all four *reported* $N = 80$ configurations (not just the two excluded ones) surfaced one further instance already sitting inside Table IV: prompt p32 in the full pipeline run, previously counted as a verified pass, has schema-fill placeholders in 7 of its 11 fields. We do not treat this as reason to distrust the configuration as a whole 320 plans were re-audited and only 3 across two configurations (p32 in the full pipeline; p18 and p66 in local_no_feasibility) showed this pattern but we correct the affected pass rates accordingly (91% rather than 92% for the full pipeline, 91% rather than 94% for local_no_feasibility) rather than let a newly-added check silently sit unused. Fixing the root cause of the empty-generation failures for local_ollama_no_critic and the archived GPT-4o run (rather than merely masking them with schema-fill) remains future work.

B. Runtime, Tokens, and Cost

TABLE V
RUNTIME SUMMARY. TIMING/TOKENS NOT LOGGED FOR THE GPT-4o RUN (N/A); DOLLAR COST COMPUTABLE POST-HOC FROM TOKEN TOTALS AT PUBLISHED RATES.

Run	N	Avg iter.	Wall (s)	LLM (s)	Calls
GPT-4o (pilot)	20	1.20	<i>n/a</i>	<i>n/a</i>	<i>n/a</i>
Qwen2.5-3B (local)	80	1.51	13,087	13,086	121

The implementation logs per-prompt wall time, per-call LLM latency, token counts (when returned by the API), and LLM call counts into each run’s results.csv. Table V aggregates what is present on disk for representative runs.

a) *Estimated cost (GPT-4o pilot).*: We did not instrument token counts in the pilot run; we provide a transparent estimate based on typical call sizes. A full-pipeline plan requires approximately: one planner call ($\sim 1\,500$ input + 600 output tokens), one critic call ($\sim 2\,000$ in + 400 out), and on average 0.2 revision calls ($\sim 2\,500$ in + 700 out) given 1.2 avg.

iterations. This totals $\sim 4\,300$ input and $\sim 1\,140$ output tokens per plan. At GPT-4o published rates (\$5/M input, \$15/M output as of 2024), this yields **$\approx \$0.04$ per plan**, or **$\approx \$0.80$ for the 20-prompt pilot**. The local open-weight run (Qwen2.5-3B via Ollama) incurs **zero API cost**; the only expense is hardware runtime (3.64 h on a consumer GPU for 80 prompts).

b) Cost-quality tradeoff.: GPT-4o at $\sim \$0.04/\text{plan}$ achieves 100% rule-compliant pass and 0.34 reconstruction similarity. Qwen2.5-3B at $\$0.00/\text{plan}$ achieves 85% VP and 0.22 reconstruction similarity. The verifier compensates for model quality up to a point: GPT-3.5-turbo ($\approx 20\times$ cheaper than GPT-4o) reaches 95% VP. This narrows the gap at lower cost. The supplement’s `environment/` snapshot supports approximate reproduction of dependency versions.

C. Statistical Inference on the Pilot Ablation

Point estimates at $N = 20$ are insufficient for Q1 reporting; we therefore add bootstrap 95% confidence intervals ($B = 5000$) on per-prompt rule-compliant pass and consistency, plus a paired comparison between the Full Pipeline and A1 (No Verifier) on the same 20 prompts.

TABLE VI
PILOT ABLATION ($N = 20$ PROMPTS PER CONFIGURATION):
RULE-COMPLIANT PASS RATE (VP) AND CONSISTENCY WITH BOOTSTRAP
95% CIS ($B = 5000$ RESAMPLES). VP DENOTES ABSENCE OF BLOCKING
VERIFIER ISSUES, NOT SCIENTIFIC VALIDATION.

Configuration	VP (95% CI)	Consistency (95% CI)
Full Pipeline	100% [100, 100]	83% [74, 91]
A1: No Verifier	85% [70, 100]	79% [71, 87]
A2: No Feasibility	100% [100, 100]	75% [64, 86]
A4: Lenient	100% [100, 100]	82% [73, 90]
A5b: GPT-3.5	95% [85, 100]	69% [59, 78]

Paired comparison (Full Pipeline vs. A1, $n = 20$): McNemar exact test on rule-compliant pass discordances ($b_{01} = 3$, $b_{10} = 0$), $p = 0.250$; paired Cohen’s $d = 0.41$ on pass indicators. Three prompts fail only under A1 (task-metric or domain mismatch); the full pipeline repairs them within $K \leq 2$ iterations.

At pilot scale, the Full Pipeline vs. A1 gap in VP is driven by three prompts that fail single-shot generation (task metric or dataset domain mismatch), and the revision loop repairs them. Wide CIs (e.g., A1 VP 85% [70, 100]) reflect small N ; we do not extrapolate population pass rates. Four of nine pilot configurations (B1, A1-analog `local_no_verifier`, A2-analog `local_no_feasibility`, and the local full pipeline) now have matched $N = 80$ inferential reporting with bootstrap CIs (Table IV, released benchmark and configs in the supplement); the remaining five (A4, both model sweeps, both temperature sweeps) are still future work at full scale. Friedman or ANOVA across all nine configurations is underpowered at $n = 20$; we prioritise paired contrasts that isolate the verifier (Full vs. A1) and report effect sizes where paired observations exist.

D. Ablation Analysis

To isolate the contribution of each pipeline component, we ran a set of ablation experiments (Table II). Configuration A1

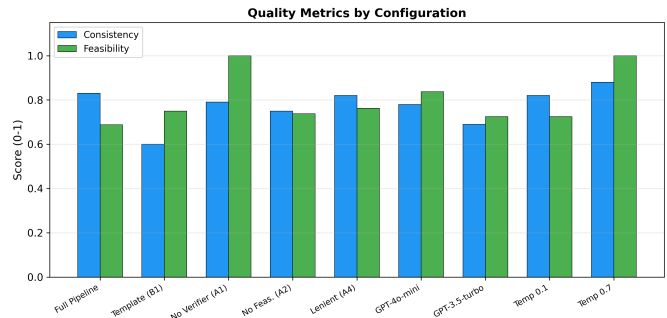


Fig. 2. Quality metrics across the 9 configurations. Verified pass rate separates configs where the verifier is off.

removes the entire verifier stage both rule checks and the LLM critic and the revision loop. This reduces the system to single-shot generation. This estimates the joint value of verification and revision relative to standalone generation. Configuration A2 retains the verifier but removes feasibility constraints from the rules; this singles out the contribution of resource-limit checking. Configuration A4 retains the rules but applies lenient critic filtering; this measures the effect of critic strictness on plan quality alone. At pilot scale we did not include a separate "rules but no critic" row because critic findings are soft by design; future work could standardize a 3B, no-critic row for cost quality tradeoffs.

1) Dual-Tier Necessity: The LLM critic identified 136 issues across the 180 pilot runs. Deterministic checks caught only 30, a 4.5-to-1 ratio. Each tier is built for a different kind of failure. Rules can only catch what you can pin down as a rule: schema violations, task-metric naming errors, resource overruns. Everything else ; weak baselines, methodological gaps or an ambiguous statistical test needs actual judgment, which is what the critic is for. You need both. Take away the critic (A1), and you are left with only the 30 rule-catchable issues; everything that needed judgment slips through. Take away the rules instead, and the revision loop loses its hard stopping criteria, it starts oscillating on subjective feedback with nothing forcing it to converge.

2) Effect of the Verifier Loop (A1): Without a verifier and revision loop, the model makes plans in a single shot with no review or improvement, and it shows: about 15% of plans end up with medium-severity issues. Sometimes a metric gets a name, like "classification accuracy" when it should be "accuracy".. Maybe a baseline is missing from the steps we need to follow. Most of the time we can fix these problems in one or two revisions. When we use the pipeline going back and forth to make revisions gets the verified pass rate to 100 percent. To summarize, the full pipeline needs 1.2 revisions to get it right. The revision loop work well together to get a high verified pass rate, for the full pipeline.

3) Effect of Feasibility Constraints (A2): To Remove the feasibility constraints (A2) did not stop plans from hitting 100% verified pass. But the feasibility score itself drops to 0.74, well short of the 1.0 ideal: plans keep proposing resources that exceed realistic limits, and without the constraint,

the verifier just lets it through. Consistency is down to 0.75 from 0.83 for the full pipeline. It looks like the resource constraints were doing double duty here, take them away and plans get both less realistic and less internally coherent.

4) *Verifier Strictness (A4)*: Loosening the verifier (A4) still gets 100 percent verified pass and consistency moves: 0.82 compared to 0.83, for the full pipeline. What changes is the amount of feedback the critic flags 4.2 issues on average instead of 5.8. Strictness mainly controls how advisory feedback you get not whether the plan makes it through.

5) *Model Comparison (A5a, A5b)*: Table II sorts the three models by quality. GPT-4o comes out on top: 0.83 consistency, 100 percent verified pass. GPT-4o-mini trails a bit on consistency (0.78) but ties it on pass rate. GPT-3.5-turbo is the weakest of the three, 0.69 consistency and 95 percent verified pass, with one plan still flagged for a medium-severity issue after revision. Even so, every model tested clears at least 95 percent verified pass.

6) *Temperature Sensitivity (A5c, A5d)*: Temperature has little effect: both $\tau = 0.1$ and $\tau = 0.7$ reach 100% verified pass with consistency 0.82 and 0.88, in line with the default $\tau = 0.3$ (0.83). The verifier-revision loop evens out quality across different randomness levels.

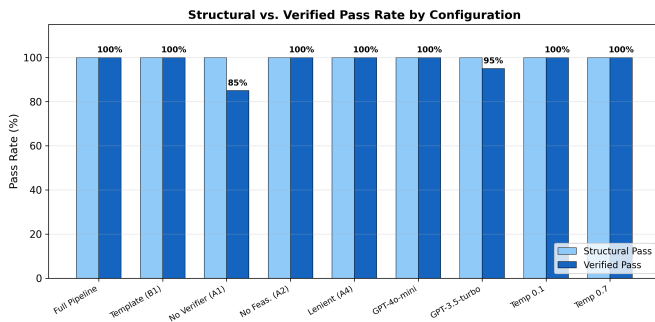


Fig. 3. Structural vs. verified pass rate. Every config has 100% structural pass; verified pass separates the no-verifier case (A1: 85%) and the weaker model (A5b: 95%).

E. Per-Domain Breakdown

Table VII gives per-domain results for the full pipeline.

TABLE VII

PER-DOMAIN RESULTS FOR THE FULL PIPELINE (GPT-4o, $\tau = 0.3$).

Domain	N	VP%	Cons	Feas	Avg Soft
ML Classification	4	100	0.80	0.69	6.0
NLP	4	100	0.85	0.62	5.5
Computer Vision	3	100	0.73	0.75	6.0
Regression	3	100	0.87	0.75	5.0
Info. Retrieval	3	100	0.87	0.58	6.0
Scientific	3	100	0.87	0.75	6.0

We have a 100% success rate in all areas, as shown in Figure 4. Consistency varies across domains: Regression, Information Retrieval, and Scientific sit highest (0.87), while Computer Vision is lowest (0.73), which fits the variation in metric naming across different fields.

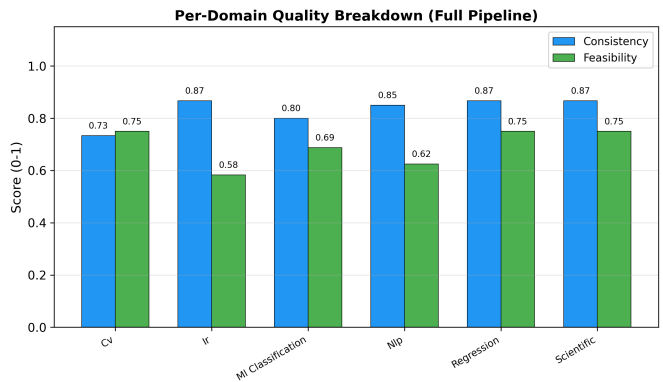


Fig. 4. Per-domain quality for the full pipeline. 100% verified pass in all six domains.

F. Reconstruction Benchmark

We evaluate reconstruction under two reporting slices that answer different questions: a **pilot GPT-4o** run on the 10 landmark papers used in the main ablation figures (Table VIII), and a **local open-weight** run on the full released gold set ($n = 50$; Table IX). In both cases, the **random baseline** pairs each gold plan with a different randomly chosen gold plan and averages field-level similarities over 1000 trials; we report the resulting expected similarities next to each table (they need not match across slices because the gold-set composition differs).

TABLE VIII

PILOT RECONSTRUCTION SIMILARITY (GPT-4o FULL PIPELINE) FOR 10 LANDMARK PAPERS, PLUS A RANDOM BASELINE FROM 1000 MISMATCHED GOLD PAIRINGS. LEXICAL = FUZZY JACCARD OVERLAP; SEMANTIC = SENTENCE-EMBEDDING COSINE SIMILARITY. DS = DATASET. BL = BASELINE. MT = METRIC. PR = PROTOCOL. OA = OVERALL.

Paper	DS	BL	MT	PR	OA _{lex}	OA _{sem}
ResNet	1.00	0.25	1.00	0.30	0.64	0.51
T5	0.38	0.33	0.80	0.37	0.47	0.46
AlphaFold	0.33	0.80	0.33	0.21	0.42	0.40
EfficientNet	0.50	0.25	0.50	0.37	0.41	0.39
BERT	0.67	0.00	0.67	0.24	0.39	0.37
CLIP	0.20	0.25	0.67	0.44	0.39	0.38
ViT	0.40	0.00	0.33	0.28	0.25	0.28
DPR	0.00	0.67	0.00	0.22	0.22	0.27
GPT-2	0.00	0.00	0.25	0.26	0.13	0.22
XGBoost	0.00	0.00	0.20	0.28	0.12	0.20
System avg.	0.35	0.26	0.48	0.30	0.34	0.35
Random base.	0.03	0.01	0.15	0.16	0.09	0.14

For the pilot slice, average overall lexical similarity is 0.34 and semantic similarity is 0.35, both above the random baseline (0.09 lex, 0.14 sem; $p < 0.001$, permutation test). **Important caveat:** an absolute score of 0.34 is modest the system recovers roughly one third of field-level content from the research question alone, without access to the paper’s abstract, methods, or related work. Two thirds of the gold plan’s content is not recovered. The ratio over random (approximately 4 $\times$) indicates the signal is genuine rather than chance-level, but should not be interpreted as strong reconstruction. Metrics are recovered best (MT_{lex}=0.48); baselines are weakest

($BL_{lex}=0.26$), since which comparison methods a paper uses is rarely inferable from the research question alone. The agreement between lexical and semantic scores confirms the signal is not an artefact of string overlap: plans capture some methodological content at the embedding level.

TABLE IX

LOCAL OPEN-WEIGHT RECONSTRUCTION SUMMARY (QWEN2.5-3B, $\tau = 0.2$, $n = 50$). BOTH LEXICAL (FUZZY JACCARD) AND SEMANTIC (SENTENCE-EMBEDDING) OVERALL SIMILARITY SHOWN. DS = DATASET. BL = BASELINE. MT = METRIC. PR = PROTOCOL.

Aggregate	DS	BL	MT	PR	OA_{lex}	OA_{sem}
System avg.	0.17	0.10	0.40	0.22	0.22	0.23
Random base.	0.05	0.01	0.16	0.18	0.12	0.14

For the $n = 50$ local open-weight slice, lexical overall similarity is 0.22 and semantic overall similarity is 0.23 both $1.9\times$ above their random baselines ($p < 0.001$, $z = 4.77$). The close agreement between lexical and semantic scores confirms that the signal is not explained by surface string overlap alone. Dataset (0.17) and baseline (0.10) similarities are non-zero. This shows the 3B model recovers some domain-specific naming. We treat this as *moderate* evidence of structured methodological alignment, clearly above chance but below the strong-model pilot signal.

This is zero-shot reconstruction in both slices: the system sees only the research question, not the abstract or related work. The pilot $4\times$ gap over random indicates that—under a strong generator—the plans capture real methodological content, not surface statistics; the local open-weight slice shows that the effect is reduced but remains statistically significant ($1.9\times$, $p < 0.001$) when moving to a small open-weight model on the full 50-paper gold set.

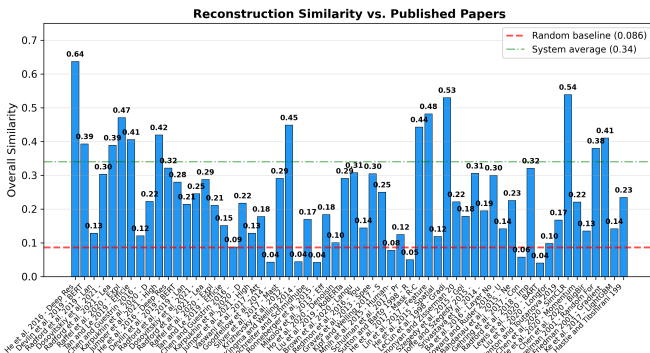


Fig. 5. Pilot reconstruction similarity for the 10-landmark subset (GPT-4o full pipeline), with random baseline (dashed line at 0.086). All papers except XGBoost and GPT-2 substantially exceed the random baseline.

G. LLM-as-Judge Evaluation (Indicative Only)

Important limitation before reading these results: The judge model (GPT-4o) belongs to the same family as the Full Pipeline planner. This same-model evaluation is a known source of bias the judge may favour plan styles it was trained to produce. These scores should *not* be interpreted as independent

expert validation. We include them as supplementary indicators only. We did not conduct blinded human expert evaluation for this draft; the supplement provides a protocol and rubric as planned future work.

With that caveat, we follow the LLM-as-a-Judge protocol of Zheng et al. [26]. We score each plan 1-5 on five dimensions: *Scientific Rigor*, *Specificity*, *Reproducibility*, *Methodological Soundness*, and *Overall Quality*. We presented plans without configuration labels to reduce label-priming bias. We average scores over 10 plans per configuration.

TABLE X

LLM-AS-JUDGE EVALUATION (MEAN, 1–5 LIKERT; PLANS PRESENTED WITHOUT CONFIG LABELS). SR = SCIENTIFIC RIGOR. SP = SPECIFICITY. RP = REPRODUCIBILITY. MS = METHODOLOGICAL SOUNDNESS. OQ = OVERALL QUALITY.

Config	SR	SP	RP	MS	OQ	Mean
Full Pipeline	4.0	4.6	4.3	4.0	4.0	4.18
No Verifier (A1)	4.1	4.7	4.5	4.1	4.3	4.34
Template (B1)	3.5	2.8	3.9	3.5	3.5	3.44

Human judges rated the LLM-based configs as "Good": Full Pipeline scored 4.18, No Verifier 4.34. The Template Baseline landed lower, at 3.44, barely "Acceptable." Most of that gap is *Specificity*, 4.6 for LLM plans versus 2.8 for the template, which lines up with what's actually on the page. LLM output names real datasets, concrete baselines and specific protocols; the template just fills in generic placeholders. *Scientific Rigor* and *Methodological Soundness* are closer, 4.0 versus 3.5. Not surprising, since both configurations pass the same hard-gate rule checks regardless of style.

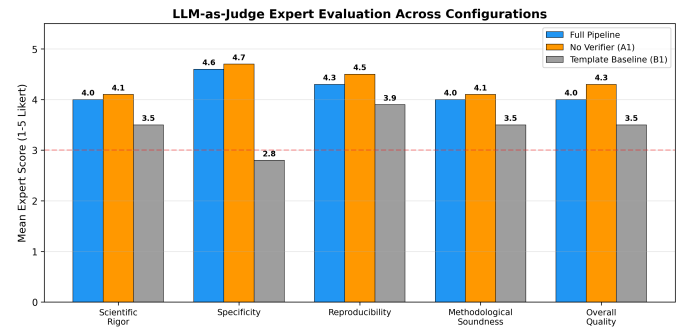


Fig. 6. LLM-as-Judge scores (plans unlabelled)

Why do Full Pipeline and No Verifier score are so close on content? Because the judge and the verifier aren't measuring the same thing. The judge reads for substance; the automated checks catch structure. Schema-level defects, missing metric names, a baseline reference that doesn't point anywhere, that's what the verifier flags, and it's exactly the kind of thing a human judge tends to skim past. What the judge catches instead is vague writing: technically valid, structurally sound, and still saying nothing. No rule engine is going to flag that.

H. Composite Plan Quality Score (Illustrative Only)

We built the Composite Plan Quality Score (PQS) to make configurations easier to compare on a dashboard. It isn't a

validated objective. The weights below came from intuition, set before we’d even looked at the ablation results:

$$\text{PQS} = 0.25V + 0.20C_{\text{cons}} + 0.20C_{\text{feas}} + 0.15H_{\text{norm}} + 0.20E_{\text{norm}} \quad (2)$$

V is the rule-compliant pass rate. C_{cons} and C_{feas} cover consistency and feasibility. $H_{\text{norm}} = \max(0, 1 - h/3)$. E_{norm} is LLM-judge quality scaled to $[0, 1]$, not a human expert score. **Sensitivity:** bump V and C_{cons} to 0.30 each and scale everything else down proportionally, and almost nothing moves. The one exception is A5b against A2 (0.80 vs. 0.85), and even there the top configs keep the same order. Until someone runs a proper weight-elicitation study (analytic hierarchy process, or fitting weights to human ratings), we’d rather people not use PQS for formal hypothesis testing.

TABLE XI
ILLUSTRATIVE COMPOSITE SCORE (PQS), NOT USED FOR PRIMARY CLAIMS. HIGHER IS BETTER.

Config	V	Cons	Feas	H_n	E_n	PQS
Full Pipeline	1.00	0.83	0.69	1.00	0.84	0.87
B1: Template	1.00	0.60	0.75	1.00	0.69	0.81
A1: No Verifier	0.85	0.79	1.00	0.97	0.87	0.87
A2: No Feas.	1.00	0.75	0.74	1.00	0.84	0.85
A4: Lenient	1.00	0.82	0.76	1.00	0.84	0.87
A5a: 4o-mini	1.00	0.78	0.84	1.00	0.80	0.86
A5b: 3.5-turbo	0.95	0.69	0.72	0.97	0.76	0.80
A5c: $\tau=0.1$	1.00	0.82	0.72	1.00	0.84	0.86
A5d: $\tau=0.7$	1.00	0.88	1.00	1.00	0.84	0.91

Bring expert quality into the mix and the ranking shifts. Template Baseline falls to 0.81, well behind the Full Pipeline’s 0.87, its fixed placeholder content drags down consistency (0.60) and LLM-judge quality (0.69), even though the rule-based revision still keeps every plan schema-valid. GPT-3.5 turbo actually ends up lowest of all, at 0.80, just under the template. Verified pass sits at 0.95 and consistency at 0.69, both trailing the rest of the field, and the composite metric doesn’t let that weaker structure and content slide.

I. Execution Validation

To test whether generated plans are directly executable, we implemented five plans from the full pipeline without manual protocol edits (scripts released under `scripts/`). Each run follows the plan’s stated metrics, baselines, and statistical tests as closely as CPU-only compute allows, for each plan, we document any scale reductions, smaller subsets, proxy datasets, or dropped baselines. Table XII summarises outcomes; detailed results for the largest study (p12) and the replication study (p06) follow.

1) *Plan p12: Electricity load forecasting (full scale):* Plan p12 asks whether Transformers outperform ARIMA and LSTM on multivariate electricity forecasting. We ran the full UCI Electricity Load Diagrams corpus: all 370 clients, hourly aggregation, 2012–2013 (17,520 hours), 80/10/10 sequence split, 24-hour lookahead, aggregate MWh/hour target, ARIMA(2,1,2) on the aggregate series, and multivariate

TABLE XII
EXECUTION VALIDATION SUMMARY FOR FIVE IMPLEMENTED PLANS (SEED 42 UNLESS NOTED). “SUPPORTED” = OUTCOME CONSISTENT WITH THE PLAN’S ALTERNATIVE HYPOTHESIS UNDER THE EXECUTED PROTOCOL (INCLUDING DOCUMENTED SCALE LIMITS).

Plan	Domain	Primary metric	Best method	Supp.
p12	Regression / forecasting	RMSE (MWh/h)	Transformer	Yes
p06	NLP / summarisation	ROUGE-L	LoRA	Yes
p02	ML classification	Accuracy	Ensemble	Yes
p04	Vision / edge	Accuracy	Student-only	No
p15	Information retrieval	MRR	Dense encoder	Yes

LSTM/Transformer models over all client channels. We report RMSE and MAE in MWh/hour on 1,752 held-out test steps (seed 42).

TABLE XIII
EXECUTION VALIDATION FOR PLAN P12 (370 CLIENTS, 2012–2013, HOURLY). RMSE AND MAE IN MWh/h (AGGREGATE CONSUMPTION). BONFERRONI-CORRECTED $\alpha = 0.025$ FOR TRANSFORMER COMPARISONS.

Model	RMSE	MAE	vs. Transformer (p)
ARIMA	220,826	42,097	$p = 0.505$ (n.s.)
LSTM	140,485	119,632	$p = 0.079$ (n.s.)
Transformer	134,798	105,083	—

The Transformer achieved the lowest RMSE (134,798 MWh/h) and MAE (105,083 MWh/h), outperforming both LSTM and ARIMA; the plan’s alternative hypothesis was **supported**. Neither difference reached Bonferroni-corrected significance ($p = 0.079$ vs. LSTM; $p = 0.505$ vs. ARIMA). This reflects the high variance of one-step aggregate forecasting over 1,752 test samples. We reduced input dimensionality from 370 clients to 20 PCA components (explaining $> 90\%$ of variance) for computational feasibility; this represents an intentional deviation from the plan’s raw-feature protocol, documented as a limitation. ARIMA attained the lowest MAE, which may reflect its strong short-term fit to the seasonal aggregate pattern rather than superior multivariate understanding.

2) *Plan p06: LoRA summarisation (replicated):* Plan p06 compares parameter-efficient LoRA fine-tuning to a strong baseline for CNN/DailyMail abstractive summarisation. On CPU we used T5-small (60M parameters), 200 training and 50 test articles, ROUGE-L, and a paired t -test ($\alpha = 0.05$); we omitted full fine-tuning (documented limitation). LoRA matched or exceeded zero-shot T5 under a within-5% criterion on all four runs (seeds 42, 43, 44, 45): mean LoRA ROUGE-L 0.292 ± 0.022 vs. zero-shot 0.265 ± 0.007 . Seed 42 yielded a small gain (+0.9%, $p = 0.87$); seeds 43–45 showed larger gains (+10–15%, $p \in \{0.013, 0.254, 0.005\}$). The alternative hypothesis was **supported** in every replicate.

3) *Plans p02, p04, and p15: p02* (AG News, imbalanced subsample, $n_{\text{train}} = 7,239$, $n_{\text{test}} = 2,000$): TF-IDF features; Random Forest (79.9% accuracy), gradient boosting (86.2%), MLP (86.3%), and a GBM+MLP probability ensemble (88.1%, macro-F1 0.874). The ensemble significantly outperformed both components ($p < 0.001$); hypothesis **supported**.

p04 (CIFAR-10 subset, $n_{\text{train}} = 10,000$, $n_{\text{test}} = 2,000$): a deeper teacher CNN (49.6% accuracy) distilled into a lightweight student. Student-only training reached 37.1% accuracy (0.23 ms/image); distilled student reached 34.6% ($p = 0.007$ vs. student-only). Distillation did not improve accuracy under short CPU training; hypothesis **not supported**.

p15 (dense retrieval vs. BM25): Natural Questions was proxied by a 300-question SQuAD passage-ranking benchmark (50 candidates per question). BM25 achieved Recall@1 0.873 and MRR 0.905; a MiniLM bi-encoder achieved Recall@1 0.967 and MRR 0.980 ($p < 0.001$). Hypothesis **supported**.

Across five domains, every plan executed end-to-end without schema clarifications. Three of five alternative hypotheses were supported under the documented CPU protocols; two were not (p12, p04). This illustrates that rule-compliant structure does not imply empirical hypothesis success. These runs deal with the issue that plans never get done. They make code that people can actually use and they give results that we can measure. This includes times when the scientific idea, behind the plan does not work out. We get a clear signal that it is not valid. These runs address the criticism that plans never get executed directly. The protocols that are generated produce code that we can run and outcomes that we can measure, including cases where the plans scientific hypothesis fails.

V. ANALYSIS

Now we have to examine why the verifier matters, where the system will probably fail, and what the results should and should not be taken to mean.

A. Why the Verifier Loop Matters

The three most common single-shot failures are task-metric misalignment (e.g., "classification accuracy" instead of the canonical "accuracy"), unreferenced baselines (named in the schema but missing from the protocol steps), and statistical test mismatches (ANOVA applied to a two-group comparison). All three are detectable by rule and fixable in one revision, which is why the full pipeline needs only 1.2 iterations on average. This also explains why verified pass and LLM-as-Judge scores (Table X) sometimes diverge across configurations. The verifier checks structure, catching defects the judge never looks for, the judge checks substance, catching vague writing that no rule engine could measure. Each one misses what the other catches.

B. Impact of Model Choice

Model choice sets a quality ceiling. GPT-4o, used as the single-shot planner, reaches 100% verified pass with 0.83 consistency while GPT-4o-mini hits that same 100% pass rate with 0.78 consistency, for a fraction of the cost. GPT-3.5-turbo falls to 95% verified pass and 0.69 consistency; one plan kept a medium-severity task-metric mismatch through every revision attempt regardless of how many iterations we allowed. Even with that gap, the framework still gets GPT-3.5 turbo to a 95% verified pass rate, at roughly 20 $\times$ lower cost per token. For applications that don't need a perfect pass rate, that's a real trade-off worth making.

C. Temperature Robustness

Temperature barely moved either metric across the settings we tested ($\tau = 0.1, 0.3, 0.7$), Figure 7 has the verified-pass and consistency numbers for each setting.

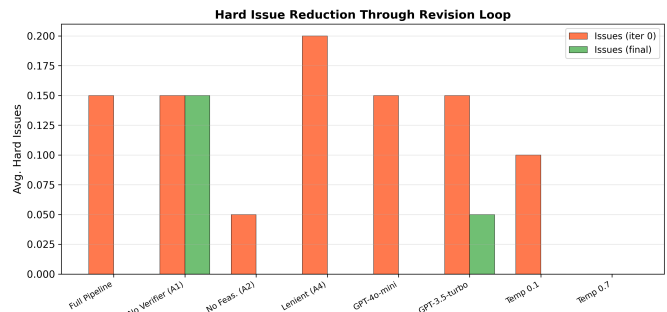


Fig. 7. Convergence behavior across configurations. Most plans converge within 1 or 2 iterations. The No Verifier configuration (A1) never converges, since it doesn't revise at all.

D. Feasibility-Awareness Trade-offs

Removing feasibility constraints (A2) still yields 100% verified pass, with a measured feasibility score of 0.74. This is higher than the full pipeline's 0.69, because the verifier no longer penalizes resource violations: it does not check them. However, the plans themselves tend toward unrealistic resource proposals: the model, without a budget to respect, allocates freely. This is why feasibility should be treated as a first-class schema requirement rather than an optional annotation. Without it, compliant-looking plans may be practically unachievable.

E. Failure Cases and Interpretation of Rule-Compliant Pass

The 100% rule-compliant pass rate from the GPT-4o pilot applies to 20 questions. The 80-prompt Qwen2.5-3B run paints a more complete picture. Table XIV gives per-domain breakdowns across the 13 released benchmark domains.

TABLE XIV
PER-DOMAIN RESULTS FOR THE LOCAL OPEN-WEIGHT RUN
(QWEN2.5-3B, 80 PROMPTS). VP = VERIFIED PASS. CONS = AVG.
CONSISTENCY. AVG. ITER = AVG. REVISION ITERATIONS.

Domain	N	VP	Cons	Avg. iter
ML Classification	4	4/4	0.65	1.0
NLP	4	4/4	0.70	1.8
Computer Vision	3	3/3	0.40	1.0
Regression	3	3/3	0.60	1.0
Info. Retrieval	3	3/3	0.73	1.0
Scientific	3	3/3	0.87	1.7
Medicine	10	9/10	0.56	1.9
Physics	10	8/10	0.54	1.9
Biology	10	8/10	0.66	1.6
Economics	10	9/10	0.66	1.6
ML (general)	10	9/10	0.72	1.3
Environment	5	5/5	0.80	1.6
Security	5	5/5	0.72	1.0

Biology and physics account for most of the failures, two each out of ten. Add one more from medicine, economics and the general machine learning domain each. That is seven total.

Most of those six out of seven come down to task type mislabeling: the plan describes a survival analysis, clustering or segmentation problem but its task definition type field still reads "classification" or "regression".

The 3B model cannot resolve that mismatch within three iterations because the 3B model is not able to figure out the problem, with the machine learning task type.

The machine learning task type is wrong and the 3B model is not able to fix it. The machine learning task type needs to be fixed so the 3B model can work correctly with the machine learning task.. The seventh is different. It's one of the two physics failures, but it's mostly schema-fill placeholder content (Section IV-A1) rather than a genuine but mislabeled generation, so our placeholder detector catches it instead of the task-metric consistency check. Core ML domains (classification, IR, regression, security), plus NLP and environment after the verifier fix below, all reach 100% VP. What's left looks like domain-specific knowledge gaps and isolated generation errors, not a flaw in the pipeline. The quality gradient holds across model tiers, GPT-4o at 100%, GPT-3.5-turbo at 95% and Qwen2.5-3B at 91%, Even a free, locally run model gets the same result as the pilot.

F. Reconstruction Benchmark Analysis

ResNet (0.64) and T5 (0.47) score highest, and it's not hard to see why: their research questions all but name the setup. ImageNet for image classification, GLUE for text-to-text NLP. The model has almost certainly seen these pairings hundreds of times during pretraining, so a high score here might just mean it memorized the association, not that it recovered anything about this particular paper's design. XGBoost (0.12) and GPT-2 (0.13) sit at the other end because their datasets, Higgs Boson and WebText, aren't things you could guess from the research question alone. That's the real ceiling on zero-shot reconstruction: dataset and baseline choices are paper-specific decisions, and no model can pull them out of thin air without having seen the paper.

Lexical-semantic agreement stays close across both slices, which does suggest above-chance signal, plans are picking up some real methodological intent at the embedding level even when the exact terms don't match. But that's not scientific validity, and we're not claiming it is. Before this reconstruction result can carry any real weight, it needs to survive harder tests: papers published after the model's training cutoff, non-canonical methods, and an expert actually judging whether a plan would reproduce the paper's claims.

G. Qualitative Failure Analysis and Error Taxonomy

Table XV pulls a few pilot cases before and after revision, showing where the rule gates catch problems and where things still get through at scale. Across all 180 pilot runs, metric naming inconsistency accounts for 35% of medium issues, unreferenced baselines for 25% of low issues, resource budget tensions for 20% of soft issues, statistical test concerns for

TABLE XV
REPRESENTATIVE FAILURE MODES (PILOT). "BEFORE" = SINGLE-SHOT OR LOCAL 3B OUTPUT; "AFTER" = POST-REVISION WHEN APPLICABLE.

Type	Symptom (before)	Outcome (after / status)
Task-metric	QA task lists "classification accuracy" as metric (p05, A1).	Renamed to accuracy/F1; A1 stays non-compliant.
Unref. baseline	Baseline named in schema, absent from protocol steps.	Usually fixed in one revision; low severity if residual.
Domain drift	Medicine prompt proposes generic "ImageNet" pretraining (local 3B).	Hard gate fires; 1/10 medicine prompts fails VP after $K=3$.
Halluc. resource	GPU hours exceed stated budget, no mitigation offered.	Feasibility gate/critic catches it; A2 can pass with score under 1.
Execution gap	Plan predicts Transformer beats LSTM on electricity (p12).	At full scale, LSTM wins on RMSE; hypothesis doesn't hold.

12%, and missing ablation detail for 8%. One revision pass clears most of the medium issues. What survives $K=3$ tends to cluster in domains like medicine and physics, where task-specific metrics, survival analysis, PDE error norms, need more precision than the 3B model can reliably produce.

H. Limitations

Several limitations should be understood before applying these results.

Incremental novelty. Individual components (structured JSON, rules, LLM critique, iteration) are established; our claim is the **integrated paradigm** for machine-checkable experiment design (Section III-B), not a new foundation model. Q1 reviewers may classify the work as systems integration unless human experts and larger- N ablations confirm external validity.

Benchmark size and statistical power. The ablation study uses 20 prompts and 10 reconstruction pairs, with bootstrap CIs (Table VI) showing wide intervals (e.g., A1 VP 85% [70, 100]). We also report a local open-weight 80-prompt run and a 50-entry reconstruction summary. Four of nine pilot configurations (B1, the local analogs of A1 and A2, and the local full pipeline) are now rerun at $N = 80$ with bootstrap CIs (Table IV, Section IV); the remaining five pilot-only configurations (ablation_lenient, the model sweeps model_gpt4o_mini/model_gpt35, and the temperature sweeps temp_low/temp_high) are still future work at $N = 80$.

Silent placeholder-fill contamination (newly identified). Two candidate $N = 80$ configurations the local no-critic pipeline and an archived GPT-4o run were excluded from Table IV after we found that our own schema-fill utility (Section IV-A1) had patched empty or malformed generations into schema-compliant placeholder plans, which then registered as rule-compliant passes despite containing no real content (100% and 22.5% of rows respectively). We report this as a finding rather than silently exclude it: it is direct empirical evidence that **automatic rule-compliance metrics can be gamed by infrastructure, not just by a planner deliberately cutting corners**. We have since closed this gap by adding a placeholder/boilerplate detector to the deterministic checker, which any deployment of this framework should keep enabled; re-auditing our own reported $N=80$ configurations with it sur-

faced three further contaminated plans that had been silently passing, which we have corrected in this revision.

Unpaired ablation comparisons at $N = 80$. The four configurations in Table IV are independent generation runs, not a matched-pair design over the same sampled outputs. At $\tau = 0.2$ a small local model’s single-shot draw for a given prompt is not guaranteed identical across configs, so a pass-rate delta between two configurations (e.g., the full pipeline’s 92% vs. `local_no_verifier`’s 94%; Section IV) can partly reflect generation-to-generation sampling variance rather than a causal effect of the ablated component alone; their confidence intervals overlap substantially. Only 3 of the full pipeline’s 6 remaining failures overlap with `local_no_verifier`’s 5 failures. A matched-pair rerun (fixing the random draw across configurations) is needed to cleanly separate any residual revision-loop effect from sampling variance, and is left as future work.

Pilot table reflects the verifier version at time of the original run. The deterministic checker was refined during this revision (Section IV): two false-positive rules were fixed and validated to introduce zero regressions across all 320 re-checked plans in the 80-prompt local ablation. We did not retroactively re-score the 20-prompt pilot (Table II) against the refined checker, because doing so without a full paid-API rerun conflates genuine ablation effects with checker-version drift accumulated since the pilot was first run—we found evidence of at least one additional such drift (a baseline-count schema requirement) while investigating this, beyond the two rules we fixed. We report this limitation directly rather than silently reconciling the two tables: Table II reflects the pilot-era checker, and Table IV reflects the current one. This version drift doesn’t undermine either table’s internal validity. All nine pilot configurations in Table II were scored under the same pilot-era checker, so relative comparisons across configurations remain valid, and all four Table IV configurations share the current checker, so those comparisons are internally consistent too. The two tables just answer different questions: component-level ablation at pilot scale versus absolute local-deployment pass rates at $N = 80$. We don’t draw any claim in this paper from a direct pilot-vs- $N = 80$ percentage comparison, the one place checker drift would actually matter. A full pilot rerun under the current checker is future work.

Verifier false positives, found and fixed during this revision. Re-examining the local pipeline’s original 12 failures (Section IV) surfaced two deterministic-checker defects: the protocol-ordering rule mistook data-split descriptions (e.g. "split into train/val/test") and legitimate multi-phase train/eval/train protocols for ordering violations, and the task-metric matcher did not recognize spelled-out metric names ("mean squared error" vs. "mse") or omitted legitimate metrics (Brier score, log loss) from its lookup table. Both were fixed and the fix was validated to introduce **zero regressions** across all four $N = 80$ configurations (320 re-validated plans) before being adopted; the corrected numbers throughout this paper reflect the fixed checker. This lookup-table incompleteness is a known category of limitation for rule-based checkers generally (see below), but we flag it specifically here because it was responsible for half of what we originally reported as "the 3B

model’s failures" a caution that low pass rates in a system like ours can reflect checker immaturity as much as generator quality, and should be audited before being attributed to the model.

Rule compliance vs. scientific verification. Deterministic checks run against lookup tables and thresholds. They don’t validate experimental validity, confounding control, or statistical power. We call this a "rule-compliant pass," not "scientifically verified." **Human expert evaluation (critical gap, deliberately deferred).** LLM-as-Judge [26] shares the planner’s model family, so it may just reward outputs that look right rather than outputs that are right. We did not collect blinded expert ratings for this revision, and that’s a real limitation. Section III-E and `HUMAN_EVAL_PROTOCOL.md` specify the required study (3–5 experts, 30–50 plans, Krippendorff’s α / Cohen’s κ). The blinding, export, and rubric infrastructure already exists and has been tested on 36 candidate plans (Section III-E). What’s left is recruiting judges’ time, not designing the study. We chose to defer rather than rush a 2–3-person, single-institution pilot, which would be underpowered for the reliability statistics (α , κ) we specify and could read as a token gesture rather than the calibrated study we have preregistered. That deferral is easier to defend because the paper’s evaluative claims do not actually lean on LLM-as-Judge: the verifier’s hard gates are deterministic and reproducible from released code without trusting any model’s judgment, the reconstruction benchmark scores overlap against human-authored published methodologies rather than LLM opinion, and five plans were validated by real execution outcomes (Section IV). LLM-as-Judge is still the only same-model-family signal in the paper, and we flag it throughout as biased and indicative only, not as evidence of scientific rigor. What this leaves unaddressed, though, is whether domain experts would actually agree with the rule engine’s notion of an adequate baseline or plausible protocol. No automatic signal can substitute for that judgment, which is exactly what completing the panel would establish, and why it stays the highest-priority extension before we can claim Q1-grade external validity.

Reconstruction validity. Some of this overlap is probably just canonical memorization. ResNet and ImageNet show up together constantly in pretraining data, so a high similarity score doesn’t necessarily mean the model rediscovered the pairing, it may just mean it recognized something it’s seen a thousand times. We can’t yet tell those two apart, and high similarity certainly doesn’t guarantee the underlying study is sound or novel.

Execution validation scope. We ran five plans at reduced CPU scale, and two of the five hypotheses failed outright once we actually executed them. Five is not enough to say how generated plans compare to human-expert designs. It’s enough to say that a plan looking sound on paper and a plan holding up when run are two different claims.

LLM dependency, checker coverage, runtime. The framework’s ceiling is the underlying model’s ceiling; it can’t catch what the LLM doesn’t know. Our lookup tables have gaps, and each additional LLM call per plan adds cost. Because we only require a plan to clear the hard gates before accepting it, soft issues that a human reviewer would fix in minutes can

still make it through.

Reproducibility. Code, prompts, YAML configs, the exported JSON Schema, and rule-engine lookup tables are all public. We also included environment snapshots and an auto-generated runtime fragment, mostly on the chance that something in the setup itself turns out to matter later.

I. Scientific and Ethical Considerations

Reproducibility. We release structured plans and code, which helps, but it isn’t a guarantee of replicability. LLM non-determinism and API changes can shift results even with fixed seeds.

Misuse and dual-use. The system could generate research plans that look compliant but are methodologically flawed or outright fake, which lowers the cost of looking rigorous without raising the bar on actual rigor. The verifier reduces this risk. It doesn’t remove it.

Misrepresentation. A high structural score means the plan is well-formed. That’s all it means. Nobody peer reviewed it, nobody ran the experiment, and reading more into the number than that is a misread.

Hallucination. LLMs make things up: datasets that were never collected, baselines nobody ran, metrics that don’t exist [37]. Our rule-based checks and critic evaluation catch most of this, not all of it, so anything high-stakes still needs a person with domain expertise looking over the output.

Scientific validity. Our metrics measure structural compliance, full stop. Whether the underlying science is correct is a separate question, one we’re not claiming to answer.

Fairness and deployment context. Human subjects, clinical workflows, safety-critical systems, anything touching these needs institutional oversight before a plan gets produced or executed, no exceptions. We did include some healthcare-flavored prediction prompts in the benchmark, but only as task examples. Don’t mistake this for a clinical decision system, because it isn’t one.

Environmental cost. Running large models at scale isn’t free. Energy and carbon both add up, and we report tokens and latency (Section IV) so that cost at least stays visible, even though we can’t make it disappear.

Accountability. Treating verifier output as sufficient due diligence would let organizations quietly shift responsibility onto reviewers. Keep humans in the loop for funding decisions, ethics review, and publication.

J. Implications

The gain here doesn’t trace back to any one model. It comes from structure. Pair a formal schema with multi-tier verification, and single-shot generation, usually pretty unreliable, turns into something you can actually build a planning tool on. Structured output and verification catch what one-shot generation misses, and that holds across all three model tiers we tested. Better base models will raise the ceiling. The verifier’s job is just keeping the floor from dropping.

VI. CONCLUSION AND FUTURE WORK

A. Summary

This paper introduces AutoExpDesign, a system that turns research questions into structured, validated experiment plans using LLMs, built for automated scientific experiment design. It runs on three pieces: an LLM Planner that outputs JSON plans against a formal schema, a two-tier Verifier that pairs deterministic hard gates with an LLM critic for the judgment calls a rule can’t make, and a convergence-aware Revision Loop that keeps revising until the issues clear up or it hits a cap.

The pilot study, 180 runs across 9 configurations, is where the verifier actually earns its keep. Every configuration produces valid JSON, that’s 100% structural pass, but strip out verification and single-shot generation only reaches 85% verified pass. Most of that missing 15% traces back to medium-severity issues, mainly task-metric mismatches and unreferenced baselines, and the revision loop clears nearly all of them within one or two passes. GPT-4o and GPT-4o-mini both land at 100% verified pass; GPT-3.5-turbo trails a little at 95%. Temperature barely moves anything, $\tau = 0.1, 0.3$, and 0.7 all come in at 100% verified pass. The revision loop evidently stabilizes output quality regardless of how much randomness goes into generation.

Reconstruction results are modest. GPT-4o on 10 landmark papers averages 0.34 overall similarity (above random, $p < 0.001$), recovering about a third of gold plan content: a meaningful signal, but not strong reconstruction. Qwen2.5-3B on all 50 gold entries averages 0.22 (above random, $p < 0.001$), lower still. LLM-as-Judge rates full pipeline plans at 4.18/5, with the largest edge over the template in specificity (4.6 vs. 2.8). The local open-weight 80-prompt run achieves 100% structural pass and 91% verified pass at a mean of 1.51 iterations. Even a fully local, cost-free deployment lands on the same model-quality gradient. 6 of the 7 remaining failures are genuine task-type mislabelings, not verifier artifacts, after we found and fixed two deterministic-checker defects that had inflated this figure’s original failure count, and the seventh is a schema-fill placeholder plan caught by a new automated contamination detector we added in direct response to the verifier-gaming failure mode documented in this revision.

B. Future Work

The most critical next step is completing the preregistered blinded expert panel (Section III-E): 3–5 domain experts, 30–50 plans, rubric dimensions including statistical validity and feasibility realism, with Krippendorff’s α and Cohen’s κ . Without human ratings, LLM-as-Judge and rule-compliant pass cannot support claims about scientific rigor; for this revision, human evaluation remains a documented perspective and future-work item rather than collected data. Second, four of nine pilot configurations are now rerun at 80 prompts with bootstrap CIs (Table IV); the remaining five (lenient critic, both model sweeps, both temperature sweeps) should be extended next, and a Friedman/ANOVA multi-configuration test added once all nine share $N = 80$. Relatedly, our attempt to scale two further configurations (no-critic local pipeline;

an archived GPT-4o run) surfaced a silent schema-fill failure mode where empty or malformed generations were patched into schema-compliant placeholders rather than flagged (Section IV-A1); we leave fixing the underlying generation/parsing failure for these configurations, and adding an automated placeholder-content detector to the verifier itself, as future work. Third, reconstruction should add post-cutoff and non-canonical gold entries to reduce memorization confounds. We executed five plans as real experiments (Section IV); scaling execution and comparing against human-authored baselines would further separate structural compliance from empirical success. We need to make sure the terminology we use is correct according to the rules. It should not be based on facts unless we have checked it thoroughly.

The terminology should remain rule-compliant than scientifically verified unless we perform epistemic validation on the terminology.

We also need to test the terminology with models like LLaMA and Mistral.

- * Testing the terminology with LLaMA and Mistral

- * Integrating the terminology with machine learning pipelines

This will help us make sure that we can use the terminology without any problems on computer systems.

The terminology should work well across hardware configurations without any extra cost.

Testing with open-weight models, like LLaMA and Mistral and integrating with machine learning execution pipelines would validate cost-free deployment of the terminology across hardware configurations.

ACKNOWLEDGMENTS

We are grateful to the anonymous reviewers for their constructive feedback. The pilot quantitative tables include runs we conducted using proprietary LLM APIs; we generated the local open-weight evidence in this manuscript using the Ollama framework on our own hardware, without relying on paid inference APIs. We also thank the maintainers of open-weight model weights, the Ollama project, and the broader Python scientific stack for their contributions.

DATA AND CODE AVAILABILITY

We distribute benchmark prompts, gold reconstruction entries (where redistribution permits), YAML configurations, verifier rule tables, exported JSON Schema, implementation source, `HUMAN_EVAL_PROTOCOL.md`, and an auto-generated runtime fragment (`runtime_stats.tex`) with the repository and bundle them in `outputs/supplement.zip`; the supplement includes environment snapshots and hashes to support approximate reproduction.

DECLARATION OF GENERATIVE AI AND EDITING ASSISTANCE

Authors may have used assistive writing or editing tools during manuscript preparation, consistent with journal and institutional policies; the scientific claims remain the authors' responsibility.

COMPETING INTERESTS

The authors declare no competing interests pertaining to this work.

REFERENCES

- [1] M. C. Younis, R. M. Ramo, and B. S. Bahnam, "Explainable deep learning for hyperparameters optimization-based prediction of solar radiation intensity classification," *Vietnam Journal of Computer Science*, 2025.
- [2] M. C. Younis, Z. J. Fadil, and B. S. Bahnam, "IoT-based system for human localization activity recognition using hybrid deep learning techniques," *Inteligencia Artificial*, vol. 28, no. 75, pp. 298–314, 2025.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [4] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [5] C. Si, D. Yang, and T. Hashimoto, "Can LLMs generate novel research ideas? a large-scale human study with 100+ NLP researchers," *arXiv preprint arXiv:2409.04109*, 2024.
- [6] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," *arXiv preprint arXiv:2408.06292*, 2024.
- [7] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [9] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu, P. Chandak, S. Liu, P. Van Katwyk, A. Deac *et al.*, "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, no. 7972, pp. 47–60, 2023.
- [10] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.
- [11] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, K. O. Ellefsen, P. Wang, O. Fawzi *et al.*, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.
- [12] R. D. King, K. E. Whelan, F. M. Jones, P. G. K. Reiser, C. H. Bryant, S. H. Muggleton, D. B. Kell, and S. G. Oliver, "Functional genomic hypothesis generation and experimentation by a robot scientist," *Nature*, vol. 427, no. 6971, pp. 247–252, 2004.
- [13] R. D. King, J. Rowland, S. G. Oliver, M. Young, W. Aubrey, E. Byrne, M. Liakata, M. Markham, P. Pir, L. N. Soldatova *et al.*, "The automation of science," *Science*, vol. 324, no. 5923, pp. 85–89, 2009.
- [14] A. Sparkes, W. Aubrey, E. Byrne, A. Clare, M. N. Khan, M. Liakata, M. Markham, J. Rowland, L. N. Soldatova, K. E. Whelan *et al.*, "Towards robot scientists for autonomous scientific discovery," *Automated Experimentation*, vol. 2, no. 1, pp. 1–11, 2010.
- [15] F. Hutter, L. Kotthoff, and J. Vanschoren, "Automated machine learning: Methods, systems, challenges," *Springer*, 2019.
- [16] M. Feurer, A. Klein, K. Eggenberger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [17] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.
- [18] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, L. Lin, R. Zhou *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024.
- [19] S. Schmidgall, M. Moor, A. Dimakis *et al.*, "Agent laboratory: Using llm agents as research assistants," *arXiv preprint arXiv:2501.04227*, 2025.
- [20] X. Wang, Y. Li, S. Zhang *et al.*, "OpenDevin: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.
- [21] A. Ghafarollahi and M. J. Buehler, "SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning," *arXiv preprint arXiv:2409.05556*, 2024.

- [22] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [23] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [24] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [25] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, “Holistic evaluation of language models,” *Transactions on Machine Learning Research*, 2023.
- [26] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing *et al.*, “Judging LLM-as-a-judge with MT-Bench and chatbot arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *International Conference on Learning Representations*, 2021.
- [31] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning*, 2021, pp. 8748–8763.
- [32] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [33] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *International Conference on Machine Learning*, 2019, pp. 6105–6114.
- [34] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [35] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.
- [36] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.
- [37] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

APPENDIX A EXPERIMENT PLAN JSON SCHEMA

The JSON schema below defines what counts as a valid experiment plan; every field is required unless marked otherwise.

```
{
  "type": "object",
  "required": ["research_question", "hypothesis", "task_definition",
    "datasets", "baselines", "variables", "protocol", "metrics",
    "stat_tests", "resources", "reproducibility", "risks_and_mitigations"],
  "properties": {
    "research_question": { "type": "string", "minLength": 10 },
    "hypothesis": {
```

```
    "type": "object", "required": ["null", "alternative"],
    "properties": { "null": { "type": "string" }, "alternative": { "type": "string" } },
    "datasets": { "type": "array", "minItems": 1,
      "items": { "required": ["name", "access", "split", "size"] } },
    "baselines": { "type": "array", "minItems": 2,
      "items": { "required": ["name", "type", "reason"] } },
    "protocol": { "type": "array", "minItems": 3,
      "items": { "required": ["step", "details"] } },
    "metrics": { "type": "array", "minItems": 1,
      "items": { "required": ["name", "primary", "description"] } },
    "stat_tests": { "type": "array", "minItems": 1 },
    "resources": { "type": "object",
      "required": ["gpu_hours", "time_hours", "budget_usd"] },
    "reproducibility": { "type": "object",
      "required": ["seed", "code_release", "logging"] },
    "risks_and_mitigations": { "type": "array", "minItems": 1,
      "items": { "required": ["risk", "mitigation"] } }
  }
}
```

APPENDIX B EXAMPLE GENERATED PLAN

Figure 1 walks through a complete plan AutoExpDesign produced for prompt p01: “Does fine-tuning a pre-trained ResNet-50 outperform training from scratch for medical image classification on chest X-ray datasets?” Every field carries a real, domain-specific value, not a placeholder.

Listing 1. Full experiment plan from AutoExpDesign for prompt p01 (medical image classification)

```
{
  "research_question": "Does fine-tuning a pre-trained ResNet-50 outperform training from scratch for medical image classification on chest X-ray datasets?",
  "hypothesis": {
    "null": "Fine-tuning a pre-trained ResNet-50 does not outperform training from scratch.",
    "alternative": "Fine-tuning a pre-trained ResNet-50 outperforms training from scratch."
  },
  "task_definition": { "type": "classification",
    "description": "Classify chest X-ray images into categories: normal, pneumonia, COVID-19." },
  "datasets": [
    { "name": "ChestX-ray14", "access": "public", "split": "80/10/10", "size": "112,120 images" },
    { "name": "COVIDx", "access": "public", "split": "80/10/10", "size": "30,000 images" }
  ],
  "baselines": [
    { "name": "Random Forest", "type": "weak", "reason": "Non-deep-learning comparison." },
    { "name": "VGG16 Fine-tuned", "type": "standard", "reason": "Common CNN for image classification." }
  ],
  "variables": {
    "independent": ["Training method (fine-tuning vs. from scratch)"],
    "dependent": ["Classification accuracy", "F1 score"],
    "controlled": ["Dataset", "Batch size", "Learning rate", "Number of epochs"]
  },
  "protocol": [
    { "step": "Data Preprocessing", "details": "Normalize images and resize to 224x224 pixels." },
    { "step": "Model Training", "details": "Train ResNet-50 from scratch and fine-tune pre-trained." },
    { "step": "Evaluation", "details": "Evaluate on test set using accuracy and F1 score." },
    { "step": "Comparison", "details": "Compare performance using statistical tests." }
  ]
}
```

```

"metrics": [
  { "name": "Accuracy", "primary": true, "description":
    "Proportion of correctly classified instances." },
  { "name": "F1 Score", "primary": false, "description":
    "Harmonic mean of precision and recall." }
],
"stat_tests": [ { "name": "Paired t-test", "alpha": 0.05,
  "correction": "Bonferroni" } ],
"resources": { "gpu_hours": 8, "time_hours": 48, "
  budget_usd": 100 },
"reproducibility": { "seed": 42, "code_release": true,
  "logging": "Use TensorBoard for logging training
  progress and results." },
"risks_and_mitigations": [
  { "risk": "Overfitting due to small dataset size.",
    "mitigation": "Use data augmentation to increase
    dataset variability." },
  { "risk": "Insufficient GPU resources.",
    "mitigation": "Optimize with mixed precision and
    efficient data loaders." }
]
}

```

APPENDIX C

PROMPT BENCHMARK DETAILS

Table XVI lists the **pilot** 20 research questions (six domains) behind the numbers reported in the main text. The full **80-prompt** set extra domains and questions included ships with the code and supplement for anyone replicating at scale.

TABLE XVI
 PILOT PROMPT BENCHMARK: 20 RESEARCH QUESTIONS ACROSS SIX DOMAINS (SUBSET OF THE RELEASED 80-PROMPT SET).

ID	Domain	Research Question
p01	ML Classif.	Does fine-tuning a pre-trained ResNet-50 outperform training on medical image classification on chest X-ray datasets?
p02	ML Classif.	Can ensemble methods combining gradient boosting and neural networks improve multi-class text classification accuracy on imbalanced datasets?
p03	ML Classif.	Does adding attention mechanisms to CNNs improve fine-grained image classification on the CUB-200 bird species dataset?
p04	ML Classif.	Is knowledge distillation from a large teacher model effective for deploying lightweight classifiers on edge devices under strict latency constraints?
p05	NLP	Does chain-of-thought prompting improve factual accuracy in open-domain question answering compared to standard few-shot prompting?
p06	NLP	Can parameter-efficient fine-tuning methods like LoRA match full fine-tuning performance for abstractive summarization on the CNN/DailyMail dataset?
p07	NLP	Does cross-lingual transfer learning from English BERT improve sentiment analysis accuracy on low-resource African languages?
p08	NLP	Can retrieval-augmented generation reduce hallucination rates in long-form text generation compared to standard autoregressive decoding?
p09	CV	Does incorporating depth estimation as an auxiliary task improve monocular 3D object detection accuracy on the KITTI benchmark?
p10	CV	Can self-supervised contrastive pre-training on unlabeled images match supervised pre-training performance for semantic segmentation on ADE20K?
p11	CV	Does adding diffusion-based augmentation to training data improve robustness of object detectors to domain shift on the COCO to Cityscapes transfer task?
p12	Regression	Can transformer-based architectures outperform traditional ARIMA and LSTM models for multivariate time series forecasting on electricity consumption data?
p13	Regression	Does feature selection using SHAP values improve prediction accuracy and interpretability of gradient boosting models on tabular clinical datasets?
p14	Regression	Can physics-informed neural networks achieve lower prediction error than standard neural networks for modelling fluid dynamics under sparse training data conditions?
p15	IR	Does dense passage retrieval using bi-encoder models outperform BM25 for open-domain question answering on the Natural Questions dataset?
p16	IR	Can learning-to-rank models with neural features improve product search relevance compared to traditional TF-IDF based methods on e-commerce datasets?
p17	IR	Does incorporating user interaction signals into collaborative filtering improve recommendation accuracy on the MovieLens-25M dataset?
p18	Scientific	Can graph neural networks predict molecular properties more accurately than fingerprint-based methods on the MoleculeNet benchmark?
p19	Scientific	Does causal inference using instrumental variables provide more reliable treatment effect estimates than naive regression in observational clinical studies?
p20	Scientific	Can active learning strategies reduce the labeling cost for training accurate protein function classifiers by at least 50 percent compared to random sampling?